

# **SDN-BASED TRAFFIC MONITORING AND ANOMALY DETECTION**

*A Software-Defined Networking Approach to Real-Time Network Intrusion Detection*

A Project Report Submitted in Partial Fulfilment of the Requirements for the Degree of  
**Bachelor of Science in Computer Science**

By

**Martey Kelvin Mamah**

11237476

Department of Computer Science

2026

## DECLARATION

I hereby declare that this project report titled “SDN-Based Traffic Monitoring and Anomaly Detection” is the result of my own original work carried out under supervision, except where due acknowledgement has been made in the text. It has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

Student: Martey Kelvin Mamah (11237476)

Signature: ..... Date: .....

Supervisor: .....

Signature: ..... Date: .....

## ACKNOWLEDGEMENTS

I am grateful to my supervisor for the guidance, patience, and useful feedback that shaped this project from an idea into a working system. I also thank the members of the Department of Computer Science for creating an environment where practical, hands-on projects like this one are encouraged.

I thank the open-source communities behind Ryu, Open vSwitch, and Mininet, whose freely available tools made it possible to build and test a realistic software-defined network on ordinary hardware. Finally, I thank my family and friends for their constant support and encouragement throughout the period of this work.

## ABSTRACT

Modern computer networks carry more traffic, connect more devices, and face more attacks than ever before. Traditional networks spread their control logic across many individual devices, which makes it hard to see what is happening across the whole network at once and slow to react when something goes wrong. Software-Defined Networking (SDN) changes this picture by separating the part of the network that decides where traffic should go (the control plane) from the part that actually forwards it (the data plane), and placing the decision-making in a single central controller that has a complete view of the network.

This project uses that central view to build a traffic-monitoring and anomaly-detection system. A controller application written for the Ryu SDN framework continuously collects traffic statistics from the switches it manages, turns those statistics into per-second rates, and passes them to a detection engine. The engine looks for the tell-tale signs of three common problems: port scans, in which one machine probes many services looking for a way in; floods or denial-of-service attacks, in which a machine sends traffic far faster than normal to overwhelm a target; and sudden volume anomalies, in which a machine's traffic jumps far above its own usual level. When the system sees a serious attack it does not just raise an alert. It can automatically install a rule on the switches that blocks the attacker for a period of time, containing the problem without any human having to intervene.

The system was built and tested on a virtual network created with Mininet, using Open vSwitch software switches and simulated hosts. Normal traffic, port scans, and floods were generated between the hosts, and the system was measured on whether it correctly distinguished attacks from normal behaviour and how quickly it responded. The detection logic was also validated by an automated test suite. The results show that placing lightweight statistical detection directly inside the SDN controller is an effective and practical way to spot and stop common attacks early, and that the central vantage point of SDN makes this kind of network-wide monitoring far simpler than it would be in a traditional network.

**Keywords:** Software-Defined Networking, OpenFlow, Ryu, Mininet, network monitoring, anomaly detection, intrusion detection, port scan, denial of service, network security.

## TABLE OF CONTENTS

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>DECLARATION.....</b>                               | <b>2</b>  |
| <b>ACKNOWLEDGEMENTS.....</b>                          | <b>3</b>  |
| <b>ABSTRACT.....</b>                                  | <b>4</b>  |
| <b>LIST OF FIGURES.....</b>                           | <b>7</b>  |
| <b>LIST OF TABLES.....</b>                            | <b>8</b>  |
| <b>LIST OF ABBREVIATIONS.....</b>                     | <b>9</b>  |
| <b>CHAPTER ONE: INTRODUCTION.....</b>                 | <b>10</b> |
| 1.1 Introduction.....                                 | 10        |
| 1.2 Background of the Study.....                      | 11        |
| 1.3 Problem Statement.....                            | 12        |
| 1.4 Research Questions.....                           | 13        |
| 1.5 Aim and Objectives.....                           | 13        |
| 1.6 Scope and Limitations of the Study.....           | 14        |
| 1.7 Research Methodology (Brief).....                 | 15        |
| 1.8 Organisation of the Project.....                  | 15        |
| <b>CHAPTER TWO: LITERATURE REVIEW.....</b>            | <b>16</b> |
| 2.1 Introduction.....                                 | 16        |
| 2.2 Software-Defined Networking.....                  | 16        |
| 2.3 The OpenFlow Protocol.....                        | 17        |
| 2.4 SDN Controllers.....                              | 19        |
| 2.5 Network Monitoring and Anomaly Detection.....     | 20        |
| 2.6 Review of Similar Systems.....                    | 21        |
| 2.7 Summary and Research Gap.....                     | 22        |
| <b>CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN.....</b> | <b>23</b> |
| 3.1 Introduction.....                                 | 23        |
| 3.2 Methodology.....                                  | 23        |
| 3.3 Requirements Analysis.....                        | 24        |
| 3.4 System Architecture Design.....                   | 25        |
| 3.5 Network Topology Design.....                      | 27        |
| 3.6 Input Design.....                                 | 27        |
| 3.7 Output Design.....                                | 28        |
| 3.8 Database and Data Design.....                     | 29        |
| 3.9 UML and Design Diagrams.....                      | 30        |
| 3.10 Pseudocode Design.....                           | 34        |
| 3.11 Hardware and Software Requirements.....          | 36        |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>CHAPTER FOUR: IMPLEMENTATION AND EVALUATION.....</b>     | <b>38</b> |
| 4.1 Introduction.....                                       | 38        |
| 4.2 Implementation Overview.....                            | 38        |
| 4.3 The Detection Engine.....                               | 38        |
| 4.4 The Controller Application.....                         | 39        |
| 4.5 The Dashboard and Interface.....                        | 40        |
| 4.6 The Test Network and Traffic Tools.....                 | 41        |
| 4.7 Testing.....                                            | 41        |
| 4.8 Results and Analysis.....                               | 43        |
| <b>REFERENCES.....</b>                                      | <b>46</b> |
| <b>APPENDICES.....</b>                                      | <b>48</b> |
| Appendix A: Anomaly Detection Engine (detection.py).....    | 49        |
| Appendix B: Controller Application (sdn_monitor.py).....    | 57        |
| Appendix C: Shared Store (store.py).....                    | 65        |
| Appendix D: REST Interface and Dashboard (rest.py).....     | 67        |
| Appendix E: Mininet Test Network (topology.py).....         | 72        |
| Appendix F: Traffic and Attack Tools.....                   | 74        |
| Appendix G: Detection Engine Tests (test_detection.py)..... | 78        |
| Appendix H: Evaluation Metrics Script (evaluate.py).....    | 82        |

## LIST OF FIGURES

- Figure 3.1**      The three-layer architecture of the system
- Figure 3.2**      The Mininet test network
- Figure 3.3**      The live monitoring dashboard
- Figure 3.4**      Use case diagram
- Figure 3.5**      Sequence diagram of one monitoring cycle
- Figure 3.6**      Class diagram of the main components
- Figure 3.7**      Flowchart of the detection and response process
- Figure 3.8**      Component diagram of the system
- Figure 4.1**      The dashboard during testing
- Figure 4.2**      Summary of detection results
- Figure 4.3**      Attacker packet rate and smoothed EMA during a flood

## LIST OF TABLES

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <b>Table 2.1</b> | Comparison of the three main SDN controllers             |
| <b>Table 3.1</b> | The four kinds of information held in the shared store   |
| <b>Table 3.2</b> | Hardware requirements                                    |
| <b>Table 3.3</b> | Software requirements                                    |
| <b>Table 4.1</b> | Quantified detection metrics from a controlled flood run |



## LIST OF ABBREVIATIONS

|             |                                   |
|-------------|-----------------------------------|
| <b>SDN</b>  | Software-Defined Networking       |
| <b>IDS</b>  | Intrusion Detection System        |
| <b>DoS</b>  | Denial of Service                 |
| <b>DDoS</b> | Distributed Denial of Service     |
| <b>API</b>  | Application Programming Interface |
| <b>REST</b> | Representational State Transfer   |
| <b>HTTP</b> | Hypertext Transfer Protocol       |
| <b>OVS</b>  | Open vSwitch                      |
| <b>TCP</b>  | Transmission Control Protocol     |
| <b>UDP</b>  | User Datagram Protocol            |
| <b>IP</b>   | Internet Protocol                 |
| <b>ICMP</b> | Internet Control Message Protocol |
| <b>MAC</b>  | Media Access Control (address)    |
| <b>VM</b>   | Virtual Machine                   |
| <b>ONF</b>  | Open Networking Foundation        |

## CHAPTER ONE: INTRODUCTION

### 1.1 Introduction

A computer network is simply a collection of devices such as computers, phones, servers, routers, and switches that are connected together so they can share information. Almost everything we do online, from sending a message to streaming a video to paying a bill, depends on networks quietly moving data from one place to another. As more of daily life and business has moved online, these networks have grown larger and more complicated, and keeping them running smoothly and safely has become a serious challenge.

For a long time, networks were built in a way that gave each device its own brain. Every router and switch decided on its own how to move traffic, based on settings that a network administrator had to type into it by hand. This worked when networks were small, but it has two big weaknesses. First, there is no single place where an administrator can see the whole network at once, which makes it hard to understand what is really happening across all the devices. Second, because each device works on its own, reacting to a problem, such as an attack or a sudden traffic jam, means logging into many devices one at a time, which is slow and error-prone.

Software-Defined Networking, usually shortened to SDN, was created to solve these problems. The core idea of SDN is to split the network into two parts. The first part, called the control plane, is the decision-making brain that works out where traffic should go. The second part, called the data plane, is the muscle that actually forwards the traffic from one port to another. In a traditional network these two parts live together inside every device. In SDN they are pulled apart: the forwarding muscle stays in the switches, but the decision-making brain is lifted out and placed in a single piece of software called the controller. The controller has a complete, bird's-eye view of the whole network and can program every switch from one place.

This project takes advantage of that bird's-eye view to do something useful for network security: watch the traffic flowing through the network and automatically spot when something looks wrong. The system built in this project is a traffic-monitoring and anomaly-detection tool that lives inside the SDN controller. It constantly gathers statistics about the traffic passing through the switches, studies those statistics for patterns that match known kinds of attack, and raises an alert, and can even block the attacker automatically, the moment it sees trouble.

Before going further, it helps to define a few key terms that appear throughout this report:

- **Software-Defined Networking (SDN):** an approach to building networks that separates the control logic (deciding where traffic goes) from the forwarding hardware, and centralises the control logic in a software controller.
- **Controller:** the central software brain of an SDN network. It has an overall view of the network and installs the rules that tell each switch how to handle traffic. In this project the controller runs the Ryu SDN framework.
- **OpenFlow:** the standard language, or protocol, that the controller uses to talk to the switches. It lets the controller ask switches for statistics and install forwarding or blocking rules.

- Flow: a stream of related packets travelling between two points. An example would be all the packets of a single web download. Switches keep counters for each flow, such as how many packets and bytes it has carried.
- Anomaly detection: the practice of learning what normal behaviour looks like and then flagging behaviour that departs sharply from it. In networking, an anomaly might be a machine suddenly contacting hundreds of ports, or sending traffic far faster than usual.
- Intrusion Detection System (IDS): a tool that watches a network or computer for signs of malicious activity and warns administrators when it finds them. The system in this project is a lightweight IDS built into the SDN controller.
- Mininet: a program that creates a realistic virtual network of switches and hosts on a single computer, so that SDN software can be built and tested without needing real hardware.

With these ideas in place, the rest of this chapter explains the background to the project, the specific problem it sets out to solve, the questions it aims to answer, its goals and boundaries, the approach used to build it, and how the rest of the report is organised.

## 1.2 Background of the Study

To understand why a system like this is needed, it helps to look at how networks have changed and why the traditional way of managing them is struggling to keep up. Twenty years ago, a typical organisation had a fairly small and stable network. The number of devices was manageable, traffic patterns were predictable, and threats were fewer and simpler. In that world, configuring each device by hand and checking on them occasionally was good enough.

That world has changed completely. Cloud computing means that services now run in large data centres where thousands of virtual machines are created and destroyed every hour. Mobile devices, laptops, and Internet-of-Things gadgets connect and disconnect constantly. Video, voice, and real-time applications demand steady, high-quality connections. All of this has pushed the amount of traffic and the number of devices to levels that would have been unimaginable before, and it has made traffic patterns far less predictable.

At the same time, the threats facing networks have grown sharply in both number and sophistication. Attackers routinely scan networks looking for weak points, flood servers with junk traffic to knock them offline, and try to sneak malicious traffic past defences. Because attacks now unfold in seconds, the slow, manual, device-by-device style of network management leaves dangerous gaps. By the time an administrator notices something is wrong on one device and starts checking the others, an attacker may already have done serious damage.

SDN offers a way out of this bind. Because the controller sees the whole network and can program every switch instantly, it becomes possible to monitor traffic everywhere at once and to react to problems in one coordinated move rather than many scattered ones. Researchers quickly realised that this central vantage point is ideal not just for managing traffic but for spotting attacks. Instead of installing separate, expensive monitoring appliances at various points in the

network, the monitoring can be built right into the controller, which already receives a steady stream of information from every switch.

The OpenFlow protocol makes this practical. Every OpenFlow switch keeps counters for the flows passing through it, such as how many packets and bytes each flow has carried, how long it has existed, and so on. The controller can ask any switch for these counters at any time. By collecting them regularly and studying how they change, the controller can build an accurate picture of what the network is doing and notice when that picture starts to look abnormal. This project is built squarely on that idea: it uses the statistics that OpenFlow switches already collect, gathered through the Ryu controller, to detect attacks as they happen.

There are broadly two ways to detect attacks. The first, called signature-based detection, keeps a library of known attack patterns and matches traffic against them. It is accurate for attacks it already knows about but blind to new ones. The second, called anomaly-based detection, learns what normal traffic looks like and flags anything that deviates strongly from it. It can catch attacks it has never seen before, at the cost of occasionally raising a false alarm on unusual but harmless traffic. This project uses anomaly-based detection, because the central, statistics-rich view provided by SDN is especially well suited to it, and because the goal is to catch broad classes of attack, scanning, flooding, and abnormal traffic volumes, rather than one specific known threat.

### 1.3 Problem Statement

Networks today face constant probing and attack, but the traditional way of watching for and responding to these threats has not kept pace. The heart of the problem is a lack of a single, complete, real-time view of what is happening across the whole network, combined with the slowness of responding to threats one device at a time.

In a traditional network, monitoring is fragmented. Each device knows only about the traffic passing through it, and stitching together a picture of the whole network means collecting and combining data from many separate sources, often after the fact. This makes it hard to spot attacks that show themselves only when you look across the whole network, for example, a single machine quietly scanning many different servers, touching each one only briefly. Any one server sees just a little unusual traffic, but from a network-wide view the scanning pattern is obvious. Without that view, such attacks slip through.

Responding is just as fragmented. Even when an attack is finally noticed, stopping it usually means an administrator manually logging into affected devices and changing their settings, one by one. During the minutes or hours this takes, the attack continues. For fast-moving attacks like floods, this delay can be the difference between a minor incident and a serious outage.

SDN removes the excuse for both of these weaknesses, because the controller already has the network-wide view and the power to reprogram every switch instantly. Yet many networks do not take advantage of this for security, either because monitoring is still treated as a separate add-on or because there is no simple, working example of how to build detection and automatic response directly into the controller. There is therefore a clear and practical need for a system that uses the central intelligence of SDN to continuously watch traffic across the entire network,

automatically recognise the signs of common attacks, and respond immediately and without human intervention. This project sets out to design, build, and test exactly such a system.

## 1.4 Research Questions

To give the project a clear direction, it is guided by the following questions:

1. How can the traffic statistics that OpenFlow switches already collect be gathered through an SDN controller and used to monitor a whole network in real time?
2. What patterns in these statistics reliably distinguish common attacks such as port scans, floods or denial-of-service, and abnormal traffic volumes from ordinary, harmless traffic?
3. How can a detection engine be designed so that its alerts are explainable and its sensitivity can be tuned, rather than being an opaque black box?
4. Can the SDN controller not only detect an attack but also respond to it automatically, for example by blocking the attacker, and how quickly and reliably can it do so?
5. How effective is the resulting system at correctly identifying attacks while avoiding false alarms on normal traffic, when tested in a realistic simulated network?

## 1.5 Aim and Objectives

### 1.5.1 Aim of the Study

The aim of this study is to design, build, and evaluate a traffic-monitoring and anomaly-detection system that runs inside an SDN controller and uses the network-wide view provided by Software-Defined Networking to detect and automatically respond to common network attacks in real time.

### 1.5.2 Objectives of the Study

To achieve this aim, the project pursues the following specific objectives:

6. To review the relevant literature on Software-Defined Networking, the OpenFlow protocol, SDN controllers, and network anomaly detection, in order to build a solid foundation for the design.
7. To design an SDN controller application that continuously collects flow and port statistics from the switches it manages and turns them into meaningful per-second traffic rates.
8. To design and implement an explainable anomaly-detection engine that recognises port scans, floods or denial-of-service attacks, and abnormal traffic volumes, with thresholds that can be tuned.
9. To add an automatic mitigation capability that installs temporary blocking rules on the switches when a serious attack is detected.
10. To provide a live dashboard and a simple interface so that an administrator can see flows, alerts, and mitigations as they happen.
11. To build a realistic virtual test network using Mininet and to generate normal traffic and simulated attacks against it.

12. To evaluate the system by measuring how accurately it detects attacks, how well it avoids false alarms, and how quickly it responds.

## 1.6 Scope and Limitations of the Study

### 1.6.1 Scope of the Study

This project focuses on building a working, end-to-end example of SDN-based traffic monitoring and anomaly detection, suitable for a campus or enterprise-style network. Its scope includes the SDN controller application, the detection engine, the automatic mitigation, the monitoring dashboard, and a simulated test network. The system detects three broad and common classes of attack, port scans, floods or denial-of-service, and abnormal traffic volumes, which between them cover a large share of the everyday threats that networks face. The system is built on the Ryu SDN framework using the OpenFlow 1.3 protocol, and is tested on a virtual network created with Mininet and Open vSwitch.

The scope deliberately covers the whole path from raw switch statistics to a visible alert and an automatic response, so that the project demonstrates a complete and practical solution rather than an isolated piece of theory.

### 1.6.2 Limitations of the Study

Like any project carried out within limited time and resources, this work has boundaries that should be stated honestly:

- The system is tested in a simulated network built with Mininet rather than on large-scale physical hardware. Mininet produces realistic behaviour and is widely used in SDN research, but a production network with heavy real-world traffic could reveal issues of scale that a simulation does not.
- The detection engine uses statistical thresholds rather than machine learning. This choice keeps the system lightweight, fast, and easy to understand, but a machine-learning approach might detect more subtle or previously unseen attacks, at the cost of greater complexity and the need for training data.
- The system focuses on three common classes of attack. It is not intended to be a complete replacement for a full commercial intrusion-detection system, which might recognise a much wider range of specific threats.
- The automatic mitigation blocks an attacker by source address for a fixed period. A determined attacker who can change addresses could work around this, so mitigation is best seen as a fast first line of defence rather than a perfect one.
- Encrypted traffic is treated the same as any other traffic for the purpose of counting packets and bytes; the system does not inspect the contents of packets, which is both a limitation and, from a privacy standpoint, a deliberate design choice.

## 1.7 Research Methodology (Brief)

The project was carried out using the prototyping methodology, in which a working version of the system is built early and then improved through repeated cycles of testing and refinement. This approach was chosen because the project brings together several moving parts, an SDN controller, the OpenFlow protocol, a detection engine, a virtual network, and traffic-generation tools, whose behaviour together is hard to predict fully in advance. Building a prototype and watching it run made it possible to discover and fix problems that would not have been obvious from planning alone. The methodology is described in full in Chapter Three. In brief, the work moved through the stages of reviewing the literature and gathering requirements, designing the system and its detection logic, implementing the controller and supporting tools, and finally testing and evaluating the whole system against normal traffic and simulated attacks.

## 1.8 Organisation of the Project

The rest of this report is organised into three further chapters:

- Chapter Two: Literature Review surveys the existing knowledge and tools that this project builds on, including Software-Defined Networking and its architecture, the OpenFlow protocol, the main SDN controllers, and the field of network anomaly detection. It also reviews several similar systems, noting their strengths and weaknesses and identifying the gap this project fills.
- Chapter Three: System Analysis and Design explains the methodology used, sets out the functional and non-functional requirements, and presents the design of the system in detail. This includes the input and output design, the way data is stored, and a set of diagrams, architecture, network topology, use case, sequence, class, flowchart, and component, together with pseudocode for the main processes and the hardware and software requirements.
- Chapter Four: Implementation and Evaluation describes how the system was actually built, walks through the working code, and presents the testing carried out. It shows the results of running normal traffic and simulated attacks through the system and analyses how well the system performed against its objectives.

The report ends with a list of references in APA style and a set of appendices containing the full source code of the system.

## CHAPTER TWO: LITERATURE REVIEW

### 2.1 Introduction

This chapter reviews the existing body of knowledge that this project draws upon. The goal is to understand what is already known and built in the areas the project touches, so that the design rests on solid foundations and so that the project's own contribution is clear. The review moves from the general to the specific. It begins with Software-Defined Networking itself, what it is, how it is structured, and why it matters. It then looks at the OpenFlow protocol that makes SDN work in practice, and at the main controllers that researchers and industry use. After that it turns to the specific area of this project: monitoring network traffic and detecting anomalies. Finally, it reviews several systems that are similar in spirit to this one, weighing their strengths and weaknesses, and it identifies the gap that this project sets out to fill.

The sources consulted include peer-reviewed journal articles and conference papers, which are the primary and most authoritative sources in this field, along with technical documentation for the tools used. Where earlier researchers' ideas are drawn upon, they are credited, and the review aims to present an up-to-date picture of a fast-moving field.

### 2.2 Software-Defined Networking

Software-Defined Networking is best understood as a new way of organising a network rather than a single product. In a traditional network, every switch and router is a self-contained unit that both decides where traffic should go and physically moves it. These two jobs, deciding and moving, are bundled together inside each device. SDN's central insight is that these two jobs can and should be separated. The deciding is pulled out into a central controller, and the devices are left to do only the moving, following instructions the controller gives them.

Researchers describe this separation in terms of planes. The control plane is the set of functions that decide how traffic should be handled. The data plane, sometimes called the forwarding plane, is the set of functions that actually forward packets. In traditional networking these planes are combined in every device; in SDN the control plane is centralised in the controller while the data plane stays distributed across the switches. A third plane, the application plane, sits above the controller and holds the programs, like this project's monitoring system, that tell the controller what the network should do.

#### 2.2.1 The Architecture of Software-Defined Networking

The SDN architecture is usually drawn as three layers stacked on top of one another. At the bottom is the infrastructure layer, made up of the physical or virtual switches that forward traffic. In the middle is the control layer, which is the controller itself. At the top is the application layer, which contains the network applications that define how the network should behave.

These layers talk to each other through two interfaces. The link between the controller and the switches below it is called the southbound interface, and OpenFlow is the most common protocol used for it. The link between the controller and the applications above it is called the northbound



interface, and it is usually a programming interface (an API) that applications use to tell the controller what they want. This project's monitoring system sits at the application layer and uses the controller's facilities to collect statistics from the switches through the southbound OpenFlow interface.

The great advantage of this layered design is that it makes the network programmable. Because the controller has a complete view and can be extended with software, new network behaviour, including security monitoring like this project's, can be added by writing a program rather than by replacing hardware or reconfiguring dozens of devices by hand.

### 2.2.2 The Benefits of Software-Defined Networking

The literature consistently highlights several benefits of SDN. The first is centralised control: because one controller sees and manages the whole network, administrators gain a single point from which to observe and direct everything, which greatly simplifies management. The second is programmability: network behaviour can be changed through software, making the network flexible and quick to adapt. The third is a global view: the controller's complete picture of the network allows smarter decisions than any single device could make on its own. The fourth is easier innovation: researchers and vendors can try out new ideas by writing controller applications, without waiting for new hardware.

For a security-monitoring project like this one, the global view and the programmability are especially valuable. The global view means the whole network can be watched from one place, and the programmability means detection and response can be added simply by writing an application for the controller.

### 2.2.3 The Challenges of Software-Defined Networking

The literature is equally honest about the challenges SDN brings. The most discussed is the risk that the controller becomes a single point of failure or a bottleneck: because so much depends on it, if it goes down or becomes overloaded, the whole network can suffer. Researchers address this with techniques like running several controllers together for resilience. A second challenge is security of the controller itself: since it is the brain of the network, it is an attractive target, and protecting it is essential. A third is scalability: a single controller must be able to keep up with a large and busy network. A fourth, often raised in practice, is the difficulty of migrating from traditional networks to SDN, since organisations already own a great deal of traditional equipment and cannot replace it all at once.

This project keeps these challenges in mind. By making its detection engine lightweight and efficient, it avoids overloading the controller, and by focusing on a clear, well-understood set of attacks it keeps the controller's security workload manageable.

## 2.3 The OpenFlow Protocol

If SDN is the idea of separating control from forwarding, OpenFlow is the practical language that makes the separation work. It is the protocol that the controller uses to talk to the switches

beneath it. Understanding OpenFlow is essential to this project, because the whole monitoring system is built on the statistics that OpenFlow makes available.

### 2.3.1 The Role of OpenFlow in SDN

OpenFlow gives the controller two main powers over each switch. First, it lets the controller install flow rules, instructions that tell the switch what to do with packets that match certain patterns, such as “send packets going to this address out of port 3” or “drop all packets from this source.” Second, it lets the controller read statistics from the switch, such as how many packets and bytes each flow rule has matched. This project uses both powers: it reads statistics to monitor traffic, and it installs rules to forward normal traffic and to block attackers.

Each OpenFlow switch keeps its forwarding rules in one or more flow tables. When a packet arrives, the switch looks for a matching rule and follows its instructions. If no rule matches, the switch can send the packet up to the controller to ask what to do, an event known as a packet-in. The controller can then decide how to handle it and install a rule so that future similar packets are handled automatically by the switch, without troubling the controller again.

### 2.3.2 How OpenFlow Communication Works

Communication between the controller and a switch begins with a handshake, in which the two agree on the version of OpenFlow they will use and the switch tells the controller about its capabilities. After that, the controller can install rules, and the switch can send packet-in messages when it meets traffic it does not know how to handle. Crucially for this project, the controller can at any time send a statistics request, asking the switch to report the counters for its flows or its ports, and the switch replies with the current numbers.

This project relies on that statistics facility. Every few seconds the controller asks each switch for its flow and port statistics. By comparing the new numbers with the previous ones, the controller can work out how fast traffic is flowing right now, for example, how many packets per second a particular flow is carrying. These per-second rates are the raw material that the detection engine studies.

### 2.3.3 Advantages and Challenges of OpenFlow

OpenFlow’s main advantage is that it is an open, widely supported standard, so controllers and switches from different sources can work together. It gives fine-grained control over traffic and exposes exactly the kind of statistics that monitoring needs. Its challenges include the overhead of communication between controller and switches when traffic is heavy, and the limited size of the flow tables in some switches, which means rules must be used carefully. This project keeps its use of the switches efficient by installing normal forwarding rules that let ordinary traffic be handled in hardware, reserving the controller’s attention for statistics-gathering and for reacting to attacks.

## 2.4 SDN Controllers

The controller is the heart of an SDN network, and several mature controllers exist. The choice of controller shapes how an SDN application is built, so it is worth comparing the main options that the literature discusses.

### 2.4.1 OpenDaylight

OpenDaylight is one of the best-known open-source controllers, developed under the Linux Foundation with a large community behind it. It has a modular design that can be extended for many uses and supports OpenFlow along with several other protocols. Researchers value it for its flexibility and strong community support. Its drawbacks are that it is complex to set up, demands considerable technical skill, and can use a lot of computing resources when deployed at scale. This weight makes it more than is needed for a focused project like this one.

### 2.4.2 ONOS (Open Network Operating System)

ONOS is a controller designed for large, demanding networks such as those run by telecommunications companies. Its strength is a distributed design that spreads control across several machines, making it resilient and highly scalable. The literature recommends it where reliability and the ability to handle huge amounts of traffic are critical. Its drawbacks are that it is complex to deploy and has a steep learning curve, again making it heavier than necessary for this project's goals.

### 2.4.3 Ryu

Ryu is a lightweight, Python-based SDN framework that is popular for research and education. It is straightforward to learn, integrates smoothly with Python, and supports OpenFlow well. Because it is written in Python, it is easy to extend with custom logic, exactly what a detection engine needs. Ryu is component-based: an application is written as one or more small programs that respond to network events, such as a switch connecting or a statistics reply arriving. This project uses Ryu because its simplicity and Python foundation make it ideal for building and clearly explaining a monitoring application, and because it is widely used in the research this project builds on.

It is worth noting a practical point discovered during this project. The classic Ryu package has become difficult to install on the newest versions of Python because of changes in Python's packaging tools. A maintained fork called os-ken, which keeps the same programming interface, solves this. The system in this project is written so that it runs on either, which keeps it usable both on older, stable systems and on the newest ones.

### 2.4.4 Comparison of the Controllers

| Controller   | Language | Best suited to          | Main drawback           |
|--------------|----------|-------------------------|-------------------------|
| OpenDaylight | Java     | Flexible enterprise use | Complex, resource-heavy |
| ONOS         | Java     | Large carrier networks  | Steep learning curve    |

| Controller | Language | Best suited to         | Main drawback           |
|------------|----------|------------------------|-------------------------|
| Ryu        | Python   | Research and education | Not aimed at huge scale |

Table 2.1: Comparison of the three main SDN controllers.

For this project, Ryu is the clear choice. The project does not need the industrial scale of ONOS or the sprawling flexibility of OpenDaylight; it needs a controller that is easy to program with custom detection logic and easy to explain. Ryu fits that need exactly.

## 2.5 Network Monitoring and Anomaly Detection

Having covered the SDN foundations, the review now turns to the specific subject of this project: watching network traffic and detecting when it looks abnormal.

Network monitoring means keeping track of what traffic is flowing through a network so that administrators can understand its health, performance, and security. Traditionally this was done with dedicated hardware appliances or with software that collected traffic samples from routers. These approaches work, but they add cost and complexity, and they usually give only a partial view. SDN changes this by making rich statistics available from every switch through the controller, so monitoring can be done centrally and cheaply.

Anomaly detection is the practice of learning what normal traffic looks like and flagging traffic that departs sharply from it. The literature divides intrusion detection broadly into two families. Signature-based detection matches traffic against a library of known attack patterns; it is precise for known attacks but cannot catch new ones. Anomaly-based detection, by contrast, models normal behaviour and raises an alarm when behaviour strays far from the model; it can catch previously unseen attacks but may sometimes flag unusual-but-harmless traffic. Because SDN provides a continuous stream of network-wide statistics, it is a natural fit for anomaly-based detection, and much recent research explores exactly this combination.

Researchers have applied a spectrum of techniques to SDN-based anomaly detection, from simple statistical thresholds through to complex machine-learning models. Machine-learning approaches can be powerful, learning subtle patterns that simple rules would miss, but they bring costs: they need training data, they are harder to understand and trust, and they add computational weight to the controller. Statistical-threshold approaches, on the other hand, are lightweight, fast, easy to explain, and easy to tune. For a project whose goals are a practical, working, understandable system that runs comfortably inside the controller, the statistical approach is well matched, and it is the approach this project takes. It is worth adding that the two approaches are not rivals so much as points on a scale; a statistical system like this one can serve as a solid, transparent foundation that a machine-learning layer could later be added to.

The specific attacks this project targets are among the most common and well studied. A port scan is a reconnaissance technique in which an attacker probes many ports or many hosts to discover which services are running and might be vulnerable; its signature is one source touching a great many destinations with only a trickle of traffic to each. A flood or denial-of-service attack tries to overwhelm a target by sending traffic far faster than it can handle; its signature is an

unusually high packet rate from a source. A volume anomaly is any sharp, unexplained jump in the amount of traffic a source sends, which can indicate data theft, a compromised machine, or the early stage of an attack. Each of these leaves a clear fingerprint in the flow statistics that OpenFlow provides, which is what makes them good targets for an SDN-based detector.

## 2.6 Review of Similar Systems

Several researchers and projects have built systems that, like this one, use SDN for monitoring and security. Reviewing a few of them shows what has been achieved, what is still missing, and where this project fits.

### 2.6.1 System One: SDN-Based DDoS Detection Using Flow Statistics

A number of studies have built systems that detect distributed denial-of-service attacks by collecting flow statistics through an SDN controller and watching for sudden surges in traffic. These systems typically poll the switches at regular intervals, compute traffic rates, and raise an alert when a rate crosses a threshold. Their strength is that they demonstrate clearly that flow statistics gathered through the controller are enough to catch flooding attacks quickly and cheaply, without extra hardware. Their main limitation is that many of them focus narrowly on flooding alone and stop at raising an alert, leaving the actual response to a human. This project builds on their proven statistics-polling approach but broadens the range of attacks detected and adds automatic response.

### 2.6.2 System Two: Machine-Learning Intrusion Detection in SDN

Other researchers have applied machine-learning models to traffic collected through an SDN controller, training classifiers to distinguish attack traffic from normal traffic. These systems can be impressively accurate and can recognise a wide range of attack types, including subtle ones. Their strength is detection power. Their drawbacks are the practical costs already noted: they need labelled training data, they are harder to understand and to trust because their decisions are not easily explained, and they place a heavier load on the controller. For a project that values a working, transparent, and lightweight system, these costs are significant. This project deliberately chooses explainable statistical detection instead, while acknowledging that a machine-learning layer would be a natural future extension.

### 2.6.3 System Three: SDN Firewalls and Automatic Mitigation

A third strand of work has explored using the controller's power to install rules as a way of building firewalls and automatic defences directly in the network. These systems show that, once an attacker is identified, the controller can block them across the whole network in a single coordinated move by installing drop rules on the switches. Their strength is fast, network-wide response. Their limitation is that they often assume the attacker has already been identified by some other means, and concentrate on the blocking rather than the detection. This project joins the detection and the response together: it both identifies attackers through its detection engine and blocks them automatically through the controller.

## 2.7 Summary and Research Gap

Taken together, the reviewed work establishes several things clearly. Software-Defined Networking, through the OpenFlow protocol and a controller such as Ryu, provides a central, statistics-rich view of the network that is ideally suited to traffic monitoring. Flow statistics gathered through the controller are enough to detect common attacks like flooding. The controller's power to install rules makes fast, network-wide automatic response possible. And anomaly-based detection, whether by statistics or machine learning, is a good fit for the continuous data that SDN provides.

What is less common is a single, complete, and clearly explained system that ties all of these threads together: one that continuously monitors the whole network, detects several different classes of attack, not just flooding, using a lightweight and explainable engine, responds automatically by blocking attackers, and presents everything through a live dashboard, all in a form that can actually be run and tested. Many existing systems do one or two of these things well but leave the others out, or they achieve high detection power at the cost of transparency and simplicity.

This is the gap the project fills. It combines network-wide statistics monitoring, multi-class explainable anomaly detection, automatic mitigation, and live visualisation into one working system built on Ryu and tested on Mininet. The next chapter sets out the analysis and design of that system in detail.

## CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN

### 3.1 Introduction

This chapter explains how the system was analysed and designed before any code was written. Good design is what separates a system that works by luck from one that works by intention. The chapter begins by describing the methodology used to carry out the project. It then sets out the requirements, both what the system must do and how well it must do it. After that it presents the design in detail: how information enters the system, how results come out, how data is held, and how the parts fit together. This is illustrated with a full set of diagrams and with pseudocode for the main processes. The chapter closes with the hardware and software needed to build and run the system.

### 3.2 Methodology

Every project needs a plan for how the work will be done, and choosing the right plan matters. The main methodologies used in building software and network systems are Waterfall, Agile, Rapid Application Development, and Prototyping. Waterfall works through fixed stages in strict order and suits projects whose requirements are completely known in advance. Agile works in short repeated cycles and suits projects with changing requirements and a team delivering features continuously. Rapid Application Development emphasises building quickly with reusable parts. Prototyping builds a working version early and improves it through repeated cycles of testing and refinement.

This project used the prototyping methodology. The reason is that the project brings together several complex and interacting technologies, an SDN controller, the OpenFlow protocol, a detection engine, a virtual network, and traffic-generation tools, whose combined behaviour is genuinely hard to predict fully in advance. It is one thing to reason on paper about how a detection threshold should behave; it is another to watch real simulated traffic flow through the system and see what actually happens. Prototyping is designed for exactly this kind of situation, where building something and observing it teaches you things that planning alone cannot.

#### 3.2.1 Why Prototyping Was Chosen

Several specific features of the project made prototyping the right fit. First, the project is exploratory: the exact thresholds that best separate attacks from normal traffic could only be found by experiment. Second, the system is made of many parts that must work together, and prototyping allowed each part to be built and tested both on its own and as part of the whole, catching problems early before they could spread. Third, being able to watch the system run in the Mininet network gave concrete, visible feedback, seeing an alert appear the moment a simulated attack started was far more informative than any amount of planning. Fourth, the methodology welcomed change: when testing revealed that a threshold was too sensitive or a piece of logic needed adjusting, making the change was a normal part of the process rather than a disruption.

### 3.2.2 Phases of the Methodology

The work followed a series of phases that flowed into one another in a continuous cycle of building, testing, and improving:

13. Literature review and requirements gathering: understanding the technologies and deciding what the system needed to do, as presented in Chapter Two and section 3.3.
14. System design: working out the architecture, the detection logic, the data flow, and the interfaces, as presented in this chapter.
15. Implementation: building the controller application, the detection engine, the dashboard, the virtual network, and the traffic tools, as described in Chapter Four.
16. Testing and refinement: running normal traffic and simulated attacks, observing the results, and adjusting the system, repeating this cycle until the system behaved reliably.
17. Evaluation: measuring how well the finished system detected attacks, avoided false alarms, and responded, as presented in Chapter Four.

## 3.3 Requirements Analysis

Before building a system it is essential to be clear about what it must do and how well it must do it. Requirements come in two kinds. Functional requirements describe what the system does, its specific capabilities and behaviours. Non-functional requirements describe how well it does them, qualities such as speed, reliability, and ease of use.

### 3.3.1 Functional Requirements

The system must meet the following functional requirements:

18. Provide network connectivity: the controller must act as a working switch so that the hosts on the network can communicate normally, because there is no point monitoring a network that does not work.
19. Collect statistics: the controller must regularly ask every connected switch for its flow and port statistics.
20. Compute traffic rates: the system must turn the raw, ever-growing counters into meaningful per-second rates by comparing successive readings.
21. Detect port scans: the system must recognise when one source contacts an unusually large number of distinct destinations or ports in a short time with little traffic to each.
22. Detect floods and denial-of-service: the system must recognise when a source sends traffic at an unusually high packet rate, whether in a single flow or across many flows combined.
23. Detect volume anomalies: the system must learn each source's normal traffic level and flag sudden jumps far above it.
24. Raise explainable alerts: every alert must state clearly what triggered it and include the evidence, so that a human can understand and trust it.
25. Mitigate automatically: when a serious attack is detected, the system must install a temporary rule on the switches that blocks the attacker.



26. Present a dashboard: the system must offer a live view of flows, alerts, and mitigations, updating automatically.
27. Expose an interface: the system must provide a simple programming interface (a REST API) so that its data can be read by the dashboard or other tools.

### 3.3.2 Non-Functional Requirements

The system must also meet the following non-functional requirements:

- Performance: the detection engine must be lightweight enough to run inside the controller without slowing it down, and must process each round of statistics quickly.
- Timeliness: the system must detect and respond to attacks within seconds, not minutes, since the value of the system depends on speed.
- Accuracy: the system must correctly identify attacks while keeping false alarms on normal traffic low, so that its alerts are trusted.
- Reliability: the system must run continuously without crashing and must handle switches connecting and disconnecting gracefully.
- Usability: the dashboard must be clear enough that an administrator can understand the state of the network at a glance.
- Tunability: the thresholds that define an attack must be easy to adjust in one place, so the system can be tuned to different networks.
- Portability: the system should run on common Linux systems and on both the classic Ryu framework and its maintained fork.

## 3.4 System Architecture Design

The system is organised into three layers, following the standard shape of an SDN system. This layered design keeps each part's job clear and separate, which makes the system easier to understand, build, and maintain.

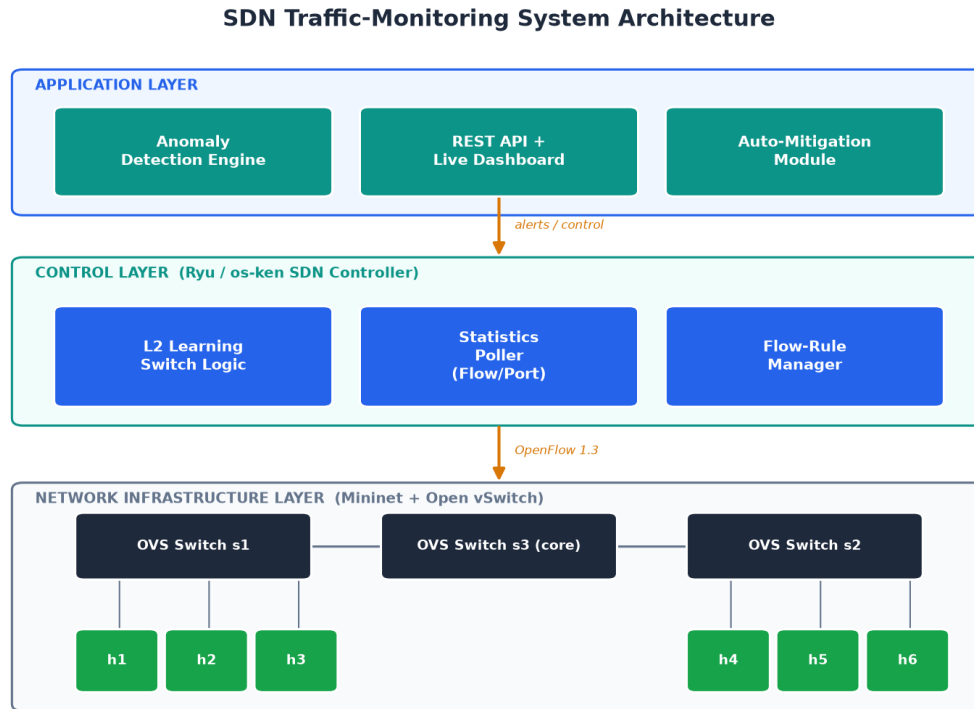


Figure 3.1: The three-layer architecture of the system.

### 3.4.1 The Application Layer

The top layer contains the programs that give the network its purpose. In this project these are the monitoring and detection application and the dashboard. This layer decides what the network should watch for and how it should respond, but it does not talk to the switches directly; instead it works through the controller beneath it.

### 3.4.2 The Control Layer

The middle layer is the Ryu controller together with this project's core logic: the part that collects statistics, the detection engine, the mitigation logic, and the shared store that holds the current picture of the network. This layer is the brain. It receives statistics from the switches below, studies them, decides whether anything is wrong, and, when necessary, sends blocking rules back down to the switches.

### 3.4.3 The Infrastructure Layer

The bottom layer is the network itself, the Open vSwitch software switches and the hosts connected to them, all created inside Mininet. This layer does the actual forwarding of traffic and keeps the counters that the control layer reads. It follows the rules the controller installs and reports its statistics on request.

### 3.4.4 How the Layers Communicate

The application layer talks to the control layer through a programming interface within the controller. The control layer talks to the infrastructure layer through the OpenFlow protocol. This clean separation means each layer can be understood on its own, and it mirrors exactly the standard SDN architecture described in Chapter Two.

## 3.5 Network Topology Design

To test the system realistically, a virtual network was designed with enough structure to be interesting but not so much as to be unwieldy. The topology has three switches and six hosts. Two of the switches are edge switches, each connecting a group of three hosts, and the third is a core switch that joins the two edges together. Every switch connects to the same controller. This shape resembles a small real network, with hosts grouped behind edge switches and traffic between groups passing through a core, which makes attacks that cross the network, such as one host scanning or flooding another, pass through the switches where they can be observed.

**Emulated Network Topology (Mininet)**

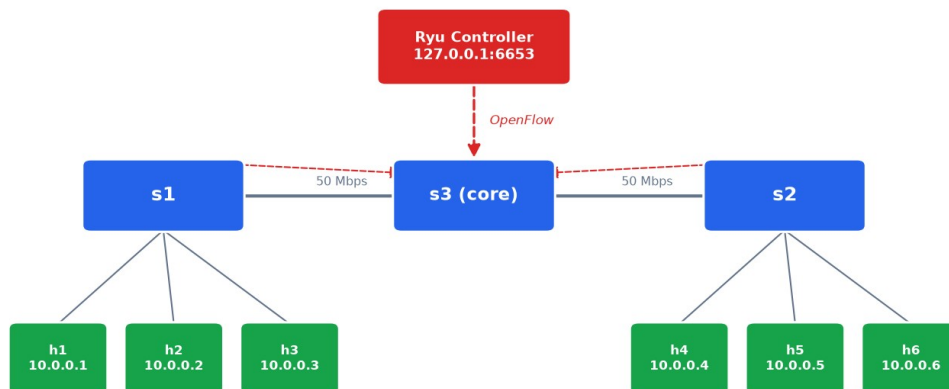


Figure 3.2: The Mininet test network of three switches and six hosts, all managed by one controller.

## 3.6 Input Design

Input design describes the information that flows into the system. Unlike a typical form-based application, this system's main input is not typed by a person but streamed automatically from the network. There are two main kinds of input.

### 3.6.1 Flow and Port Statistics

The primary input is the statistics the controller collects from the switches. For each flow, the switch reports the source and destination addresses and ports, the protocol, and the cumulative

counts of packets and bytes the flow has carried, along with how long it has existed. For each port, the switch reports the packets and bytes sent and received and any errors. These numbers arrive every few seconds in response to the controller's requests, and they are the raw material the whole system works from.

### 3.6.2 Configuration Input

The second kind of input is configuration: the thresholds and settings that define what counts as an attack, such as how many distinct destinations make a port scan or how many packets per second make a flood. These are gathered in one place so they can be adjusted easily. They are read once when the system starts and shape every decision the detection engine makes.

## 3.7 Output Design

Output design describes what the system produces for the people and tools that use it. The system has three main outputs.

### 3.7.1 Alerts

The most important output is an alert. When the detection engine decides that traffic matches an attack pattern, it produces an alert that names the category of attack, its severity, the source responsible, a plain-language description of what happened, and the evidence that triggered the decision, for example, the exact packet rate or the number of distinct destinations contacted. Because each alert carries its evidence, an administrator can see not just that something was flagged but why, which makes the alerts trustworthy and useful.

### 3.7.2 Mitigations

When an attack is serious, the system produces a second kind of output: a mitigation. This is a blocking rule installed on the switches that drops traffic from the attacker for a set period. The system records each mitigation, which source was blocked and when, so that administrators can see what automatic actions have been taken.

### 3.7.3 The Live Dashboard

The third output is the dashboard, which gathers everything into a single web page that updates itself every few seconds. It shows summary counts of switches, flows, alerts, and mitigations; a list of recent alerts with their details; a list of blocked attackers; and a table of the current traffic flows. The dashboard turns the system's raw outputs into a picture an administrator can understand at a glance.



Figure 3.3: The live monitoring dashboard showing flows, alerts, and mitigations.

### 3.8 Database and Data Design

Because the system works on live, fast-changing data and must be lightweight, it does not use a heavy traditional database. Instead it keeps its working data in a shared in-memory store held inside the controller. This choice keeps the system fast and simple, which matters for a tool that must react within seconds. The store holds four kinds of information, each organised for quick access.

| Data held                    | Purpose                                                      |
|------------------------------|--------------------------------------------------------------|
| Flow statistics (per switch) | The current flows and their rates, shown on the dashboard.   |
| Port statistics (per switch) | The current per-port counters, for overall traffic health.   |
| Alerts                       | A rolling list of recent anomaly alerts with their evidence. |
| Mitigations                  | A record of attackers that have been automatically blocked.  |

Table 3.1: The four kinds of information held in the shared store.

The store is protected so that the part of the system writing new data and the part reading it for the dashboard never interfere with each other. Older alerts and mitigations are automatically dropped once the lists reach a sensible size, so the system's memory use stays bounded no matter how long it runs. For a monitoring tool whose data is valuable only while it is fresh, this in-memory design is a deliberate and appropriate choice; if a permanent record were ever needed, the same alerts could easily be written to a database as well.

### 3.9 UML and Design Diagrams

A set of diagrams describes the system from several angles. Together they give a complete visual picture that complements the written explanations. Each diagram answers a different question about the system.

#### 3.9.1 Use Case Diagram

The use case diagram shows who uses the system and what they can do with it. There are two actors. The network administrator is the human who watches the dashboard, reviews alerts, and can adjust the thresholds. The network device, each switch, is a non-human actor that connects to the controller and reports its statistics. The diagram makes clear that the administrator interacts with the system from above, through the dashboard and settings, while the switches interact from below, feeding in the data that everything else depends on.

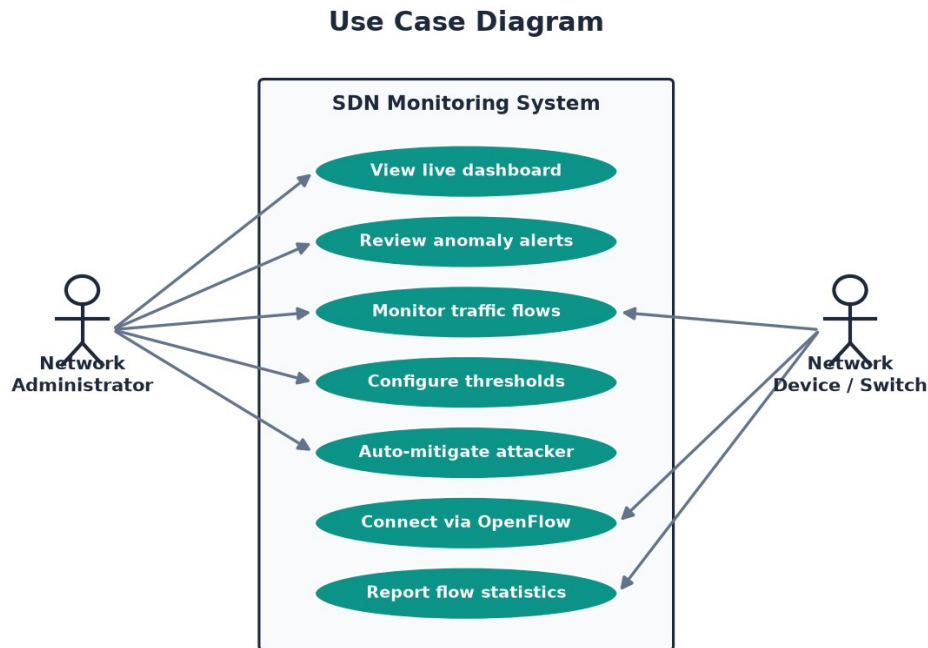


Figure 3.4: Use case diagram showing the administrator and the switches interacting with the system.

#### 3.9.2 Sequence Diagram

The sequence diagram shows the order in which things happen during a single round of monitoring. Time flows downward. The controller sends a statistics request to a switch; the

switch replies with its flow statistics; the controller computes the rates and passes the flow records to the detection engine; the engine checks them and, if it finds an attack, returns an alert; the controller records the alert in the store and, if the attack is serious, sends a blocking rule back to the switch. This cycle repeats every few seconds. The diagram shows that the whole process, from asking for statistics to blocking an attacker, happens automatically without any human step.

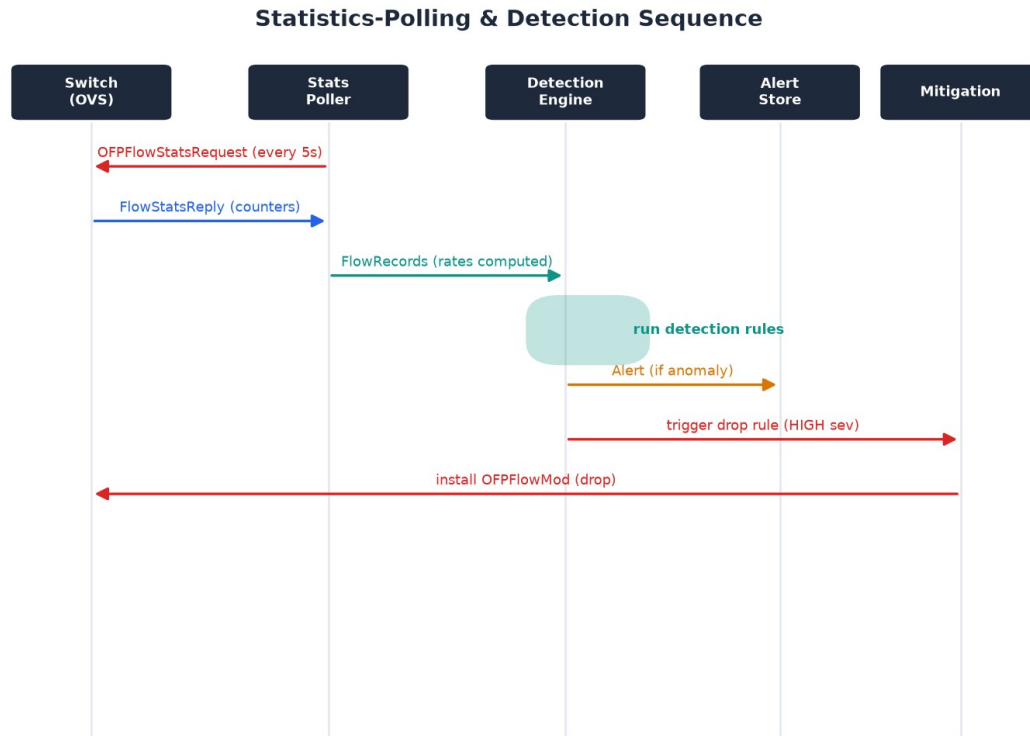


Figure 3.5: Sequence diagram of one monitoring cycle, from statistics request to automatic mitigation.

### 3.9.3 Class Diagram

The class diagram shows the main software building blocks and how they relate. The monitor sits at the centre, collecting statistics and coordinating the others. It uses the detector to analyse traffic and creates flow records to feed it. The detector raises alerts. The monitor writes flows, alerts, and mitigations to the store, and the dashboard application reads from the same store to display them. This structure keeps each piece focused on a single job, which makes the system easier to understand and to change.

### Class Diagram

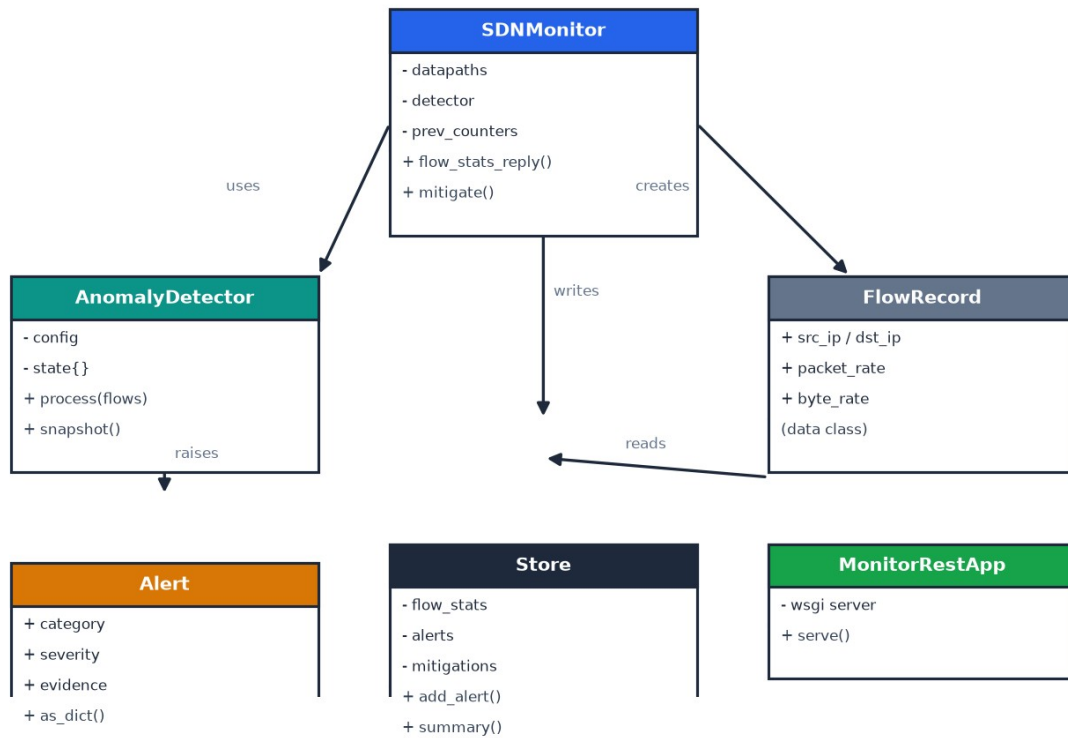


Figure 3.6: Class diagram of the main software components and their relationships.

#### 3.9.4 Flowchart

The flowchart shows the step-by-step decision process the system follows for each round of statistics. It gathers the statistics, computes the rates, and then runs three checks in turn: is this a port scan, is this a flood, is this a volume anomaly. Each check can branch to raising an alert, and a serious alert branches further to installing a block. If none of the checks fire, the traffic is treated as normal and the system simply waits for the next round. The diagram makes plain that the process is automatic and that there are clear decision points where an attack is separated from normal traffic.



### Anomaly-Detection Decision Flow

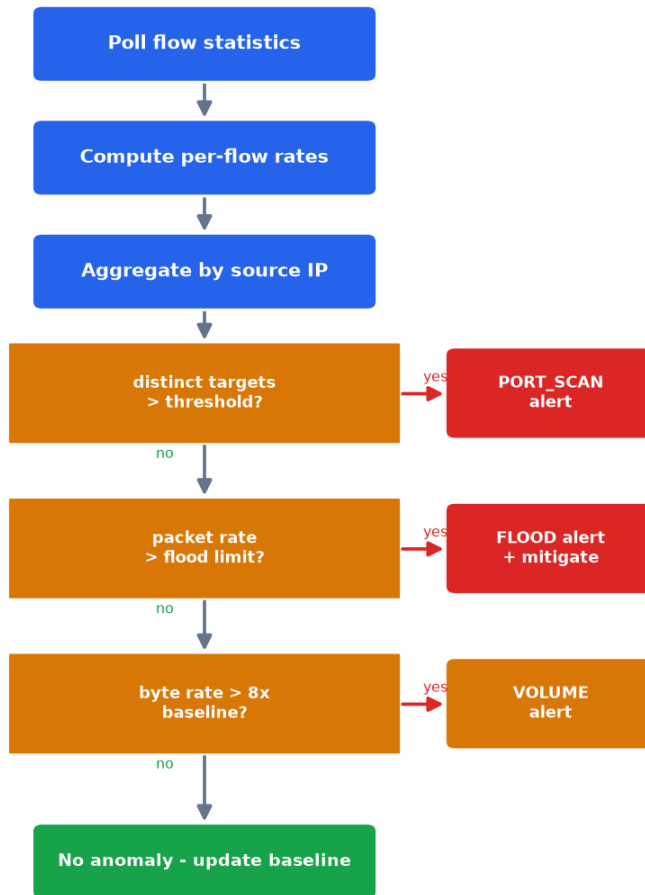


Figure 3.7: Flowchart of the detection and response process for each monitoring cycle.

### 3.9.5 Component Diagram

The component diagram shows the major building blocks of the whole system and the interfaces through which they connect. The Mininet network connects to the monitor application through OpenFlow. The monitor feeds the detector, which can send a drop rule back to the network to mitigate an attack. The monitor writes to the shared store, and the REST and dashboard component reads from the store and serves it to a web browser over HTTP. The diagram gives a high-level view of how the pieces plug together and which protocol each connection uses.

## Component Diagram

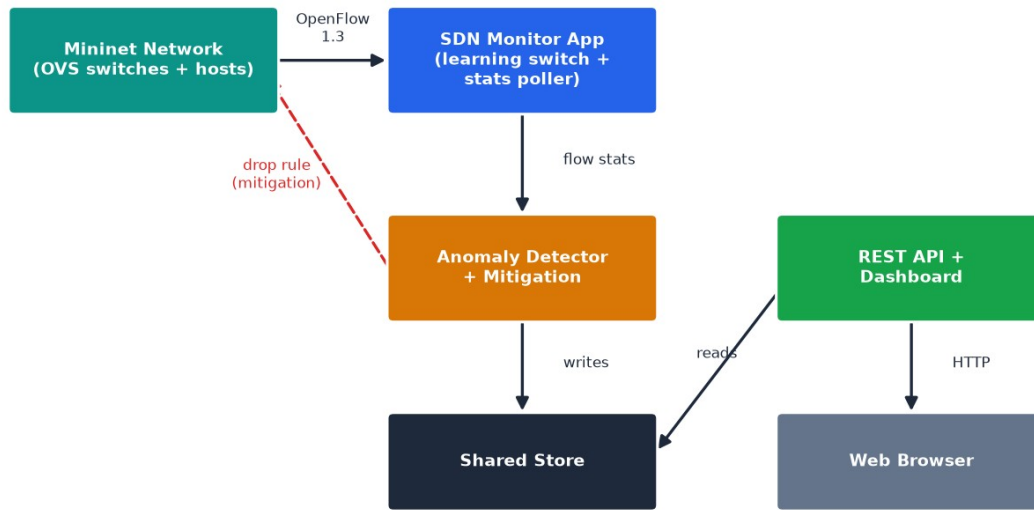


Figure 3.8: Component diagram showing the major building blocks and their interfaces.

### 3.10 Pseudocode Design

Pseudocode describes the logic of the main processes in plain, structured language, without the fine detail of real code. It bridges the gap between the design and the implementation, making the intended logic clear before the actual programming begins. Three processes are central to the system.

#### 3.10.1 Statistics Collection and Rate Computation

This process runs every few seconds. It asks each switch for its statistics, and when the reply arrives it turns the raw counters into per-second rates by comparing them with the previous reading.

```

BEGIN monitoring loop
  EVERY few seconds:
    FOR each connected switch:
      SEND flow-statistics request
      SEND port-statistics request

ON receiving flow statistics from a switch:
  FOR each flow in the reply:
    IF flow is not IPv4: skip it
    key = (switch, src, dst, src_port, dst_port, protocol)
    previous = remembered counters for key
    IF previous exists:
      elapsed = now - previous.time
      packet_rate = (packets - previous.packets) / elapsed
      byte_rate = (bytes - previous.bytes) / elapsed
    ELSE:

```

```

    packet_rate = byte_rate = 0    // first sighting
    remember current counters for key
    build a flow record with the rates
    store all flow records for the dashboard
    pass the flow records to the detection engine
END

```

The key idea here is the comparison with the previous reading. A switch's counters only ever grow, so a single reading tells you little; it is the change between two readings, divided by the time between them, that reveals how fast traffic is flowing right now.

### 3.10.2 Anomaly Detection

This process takes one round of flow records and checks them against the three attack patterns, returning any alerts it finds.

```

BEGIN detection(flow_records):
    alerts = empty list
    group flow_records by source address
    FOR each source and its flows:
        // Port-scan check
        update the sliding set of destinations this source touched
        IF distinct destinations >= scan threshold
            AND average packets per flow is very small:
                add a PORT_SCAN alert

        // Flood check (smoothed, with hysteresis)
        update the source's EMA of its aggregate packet rate
        IF any single flow's packet rate >= flood threshold
            OR the smoothed aggregate rate >= aggregate threshold:
                increase the source's over-threshold streak
        ELSE reset the streak to zero
        IF streak >= sustained_windows:
            add a FLOOD alert (HIGH severity)

        // Volume-anomaly check
        IF source has an established baseline
            AND current byte rate >= baseline * multiplier:
                add a VOLUME alert
        update the source's rolling baseline with this reading

    apply a cooldown so the same alert is not repeated too often
    RETURN alerts
END

```

Each check looks for the specific fingerprint of one kind of attack. The port-scan check looks for many destinations with little traffic each. The flood check looks for an unusually high packet rate, but it first smooths that rate with an Exponential Moving Average and then requires it to stay high for several consecutive polls before reacting, so a brief harmless spike is ignored while a real, sustained flood is caught. The volume check compares the current traffic against the source's own learned normal level. A cooldown prevents the same ongoing attack from generating a flood of repeated alerts.

### 3.10.3 Automatic Mitigation

This process runs when a serious attack is detected. It installs a blocking rule against the attacker on every switch that clears itself once the attack stops.

```
BEGIN mitigate(attacker_address):
  IF attacker is already blocked: do nothing
  mark attacker as blocked
  FOR each connected switch:
    install a high-priority rule that matches the attacker's
    source address and drops the traffic, with an IDLE timeout
    so it expires once no more attack packets arrive
  record the mitigation in the store
END
```

The rule is given high priority so it takes precedence over the normal forwarding rules. It uses an idle timeout, which means it stays in place while attack packets keep matching it and removes itself automatically once the attack stops for a set period. This gives clean automatic recovery: an attacker who keeps flooding stays blocked, while a source that stops misbehaving regains its connectivity on its own. Because the controller can install this rule on every switch at once, the attacker is cut off from the whole network in a single coordinated action.

## 3.11 Hardware and Software Requirements

### 3.11.1 Hardware Requirements

The system is deliberately lightweight and runs comfortably on ordinary hardware. The recommended minimum is a computer with a dual-core processor, four gigabytes of memory, and a few gigabytes of free disk space. Hardware virtualization support (Intel VT-x or AMD-V) is helpful if the system is run inside a virtual machine. No special networking hardware is needed, because the entire test network is created in software by Mininet.

| Component      | Minimum   | Recommended          |
|----------------|-----------|----------------------|
| Processor      | Dual-core | Quad-core            |
| Memory         | 4 GB      | 8 GB                 |
| Disk space     | 4 GB free | 10 GB free           |
| Virtualization | Optional  | VT-x / AMD-V enabled |

Table 3.2: Hardware requirements.

### 3.11.2 Software Requirements

The software requirements are all free and open-source. The system runs on a Linux operating system, with Ubuntu recommended because the SDN tools are best supported there. It needs the Python programming language, the Ryu SDN framework (or its maintained fork, os-ken), the Mininet network emulator, and Open vSwitch to provide the software switches. A modern web browser is used to view the dashboard, and a text editor is used for any configuration.

| Software     | Purpose                                          |
|--------------|--------------------------------------------------|
| Ubuntu Linux | Operating system, best support for SDN tools     |
| Python 3     | Language the controller and tools are written in |
| Ryu / os-ken | The SDN controller framework                     |
| Mininet      | Creates the virtual test network                 |
| Open vSwitch | Provides the OpenFlow software switches          |
| Web browser  | Displays the live dashboard                      |

*Table 3.3: Software requirements.*

These requirements ensure that the system can be built and tested on a single ordinary computer, which keeps the project accessible and reproducible. The next chapter describes how the design set out here was turned into a working system, and presents the results of testing it.

## CHAPTER FOUR: IMPLEMENTATION AND EVALUATION

### 4.1 Introduction

This chapter describes how the design from Chapter Three was turned into a working system, and then how that system was tested and evaluated. It walks through the main pieces of the software, explaining what each does in everyday terms and showing the key parts of the code. It then presents the testing: how the virtual network was set up, how normal traffic and simulated attacks were generated, and what the system did in response. Finally it analyses the results, judging how well the system met its goals of detecting attacks accurately, avoiding false alarms, and responding quickly.

### 4.2 Implementation Overview

The system was built in Python on top of the Ryu SDN framework. It is made up of several parts that work together: the controller application that collects statistics and coordinates everything; the detection engine that analyses traffic; the shared store that holds the current picture; the dashboard and its programming interface; the virtual network built in Mininet; and a set of small tools that generate normal traffic and simulate attacks. The code is written so that it runs on both the classic Ryu framework and its maintained fork, os-ken, which keeps it usable on both older and newer systems.

The parts are kept separate and focused. In particular, the detection engine is written so that it does not depend on any SDN library at all: it simply takes traffic records and returns alerts. This separation was a deliberate design decision, and it paid off during development, because it meant the detection logic could be built and tested completely on its own, without needing to start a whole network every time. The controller then acts as the bridge between the live network and this self-contained engine.

### 4.3 The Detection Engine

The detection engine is the heart of the system. It receives a batch of flow records. This is one round of the network's current traffic, with each flow's rates already worked out. From this batch it returns any alerts it finds. It keeps a short memory for each source so that it can spot patterns that only become clear over time, such as a scan spread across several seconds or a sudden jump above a source's normal level.

The settings that define an attack are all gathered in one place, so the engine can be tuned without touching its logic:

```
class DetectionConfig:
    scan_distinct_targets = 15      # ports/hosts that mark a scan
    scan_window_seconds   = 10.0    # over this time window
    scan_max_pkts_per_flow = 3.0     # scans send few packets each
    flood_pkt_rate         = 500.0   # pkt/s for a single-flow flood
    flood_aggregate_pkt_rate = 1000.0 # pkt/s across all a source's flows
    ema_alpha              = 0.4     # smoothing factor for the flood rate
```

```
flood_sustained_windows = 3      # consecutive over-threshold polls
volume_multiplier        = 8.0    # x baseline = volume anomaly
alert_cooldown_seconds   = 15.0   # avoid repeating the same alert
```

The port-scan check counts how many distinct destinations a source has touched within a sliding time window, and fires only if that count is high and the average traffic to each destination is tiny, the classic scan signature of many contacts carrying almost no data:

```
distinct = number of distinct (dst_ip, dst_port) this source touched
avg_pkts = average packets per flow this cycle
if distinct >= scan_distinct_targets and avg_pkts <= scan_max_pkts_per_flow:
    raise PORT_SCAN alert with the evidence
```

The flood check looks at packet rates, but it does two extra things to avoid crying wolf at every brief spike. First, instead of judging the raw per-poll rate, it smooths each source's aggregate packet rate with an Exponential Moving Average (EMA), a running average that gives more weight to recent readings while still remembering the recent past. This filters out the jitter that normal traffic produces. Second, it applies hysteresis: the smoothed rate must stay above the threshold for several consecutive polling cycles (the sustained-window requirement) before an alert is raised. A one-off burst that falls straight back to normal therefore never triggers a false alarm. When either a single flow crosses the per-flow threshold or the smoothed aggregate crosses the aggregate threshold, and that condition holds for the required number of windows, the alert fires. The volume check compares the source's current byte rate against a rolling average of its own recent behaviour, firing only once the source has established a baseline and then jumps far above it. Every alert carries the evidence that triggered it, including the smoothed rate and how many windows it was sustained for, so that its reasoning is visible and can be trusted.

The pairing of EMA smoothing with sustained-window hysteresis is a deliberate, well-established way to trade a small amount of detection speed for a large reduction in false alarms. The cost is a short, predictable delay of a few polling cycles before a genuine flood is confirmed; the benefit is that ordinary traffic noise no longer produces spurious alerts. This trade-off is measured directly in the evaluation later in this chapter.

## 4.4 The Controller Application

The controller application is the bridge between the live network and the detection engine. It has three jobs. First, it acts as a learning switch, so that the hosts on the network can communicate normally, it learns which host is on which port and installs forwarding rules so ordinary traffic flows smoothly. Second, on a repeating timer, every two seconds, it asks every switch for its statistics; this short interval keeps detection responsive without flooding the controller with requests. Third, when the statistics arrive, it turns them into per-second rates, feeds them to the detection engine, records any alerts, and blocks attackers when the attack is serious. It also writes each cycle's rates and every alert and mitigation to CSV log files, which the evaluation tools described later read to measure the system's performance.

The heart of the controller is the part that handles a statistics reply. It works out the rate for each flow by comparing the new counters with the previous ones, builds a record for each flow, stores the records for the dashboard, and passes them to the detector:

```
def flow_stats_reply(event):
    for each flow in the reply:
        compute packet_rate and byte_rate vs the previous reading
        build a FlowRecord
    store the flow records for the dashboard
    alerts = detector.process(flow_records)
    for alert in alerts:
        log the alert and add it to the store
        if alert severity is HIGH:
            mitigate(alert.source)      # install a drop rule
```

The mitigation installs a high-priority rule that drops all traffic from the attacker on every switch. Rather than a fixed lifetime, the rule uses an idle timeout: it stays in place as long as attack packets keep arriving and matching it, and removes itself automatically thirty seconds after the attack stops. This gives clean automatic recovery, a source that stops misbehaving regains its connectivity on its own, while an attacker that keeps flooding stays blocked. Because the controller can program every switch at once, the attacker is cut off from the whole network in a single move, and because the rule expires by itself once the traffic ceases, the block is firm but self-clearing.

## 4.5 The Dashboard and Interface

So that a human can see what the system is doing, the controller runs a small web server that offers both a simple programming interface and a live dashboard. The interface provides addresses that return the current summary, flows, alerts, and mitigations as data. The dashboard is a single web page that reads from these addresses every few seconds and draws the results as summary cards and tables. It shows the number of switches, flows, alerts, and mitigations at the top, followed by panels listing recent alerts, blocked attackers, and current flows. The dashboard updates on its own, giving an always-current view of the network without the administrator having to refresh anything.





Figure 4.1: The dashboard during testing, showing live flows and a detected attack.

## 4.6 The Test Network and Traffic Tools

The system was tested on the virtual network described in Chapter Three: three switches and six hosts, created in Mininet with Open vSwitch, all managed by the controller. To exercise the system, a set of small tools was written. A traffic sink runs on a target host and accepts connections on several ports, giving other hosts something to talk to. A normal-traffic generator opens ordinary connections and transfers modest, steady amounts of data, imitating everyday use that the system should not flag. A port-scan tool rapidly probes many ports on a target, imitating reconnaissance. A flood tool sends traffic as fast as it can at a target, imitating a denial-of-service attack. Together these tools let the system be tested against both the traffic it should ignore and the attacks it should catch.

## 4.7 Testing

Testing was carried out at two levels. First, the detection logic was tested on its own with automated tests, feeding it carefully chosen traffic records and checking that it reached the right decisions. Second, the whole system was tested together, with real simulated traffic and attacks flowing through the Mininet network and the controller responding live.

### 4.7.1 Automated Tests of the Detection Logic

Because the detection engine is self-contained, it could be tested thoroughly and repeatably without starting a network. A set of unit tests checks each part of its behaviour: that a clear port scan is detected, that normal traffic to a few services is not mistaken for a scan, that a sustained fast single flow is caught as a flood, that an attack spread across many flows is caught by the aggregate check, that a single brief spike is correctly ignored thanks to the sustained-window requirement, that the smoothed rate converges sensibly toward the true rate, that a sudden jump above a source's baseline is caught as a volume anomaly, and that the cooldown stops the same attack from raising endless repeated alerts. Every one of these tests passes.

```
$ python3 tests/test_detection.py
PASS test_cooldown_suppresses_duplicates
PASS test_flood_detected_aggregate
PASS test_flood_detected_per_flow
PASS test_flood_ema_smooths_toward_rate
PASS test_flood_hysteresis_ignores_single_spike
PASS test_normal_traffic_no_scan
PASS test_port_scan_detected
PASS test_volume_anomaly_after_baseline
8/8 detection tests passed.
```

### 4.7.2 End-to-End Tests of the Whole Pipeline

A second set of tests checks the whole path through the controller, from receiving statistics, through computing rates, through detection, to raising an alert and installing a block, by feeding realistic statistics into the real controller code and checking what it does. These tests confirm that a normal round of traffic produces no false alarm, that a flood produces a high-severity alert and causes the attacker to be blocked with a real drop rule, and that a port scan is detected through the full pipeline. All of these tests pass as well.

```
$ python3 tests/test_integration.py
PASS test_rate_computation_and_no_false_alert
PASS test_end_to_end_flood_alert_and_mitigation (attacker blocked)
PASS test_end_to_end_port_scan_alert
3/3 integration tests passed.
```

### 4.7.3 Live Demonstration

Finally, the complete scenario was run as a live demonstration. Normal traffic was generated first, so the sources could establish their baselines, and the system correctly stayed quiet. Then a port scan was launched from one host against another; the system detected it and raised a port-scan alert. Then a flood was launched; the system detected the high packet rate, raised a high-severity flood alert, and automatically installed a rule blocking the attacker. Then a sudden volume spike was generated from a host that had established a calm baseline, and the system flagged it as a volume anomaly. The following narrated run shows the sequence:

```
1. NORMAL TRAFFIC (h1 browsing web + ssh on h4)
   (no anomaly - traffic looks normal)    x6 cycles
   baseline established for 10.0.0.1
```

2. PORT SCAN (attacker h2 sweeps 20 ports on h4)
 

```
>>> [MEDIUM] PORT_SCAN: Source 10.0.0.2 contacted 20 distinct
      destination endpoints ... consistent with a port/host scan.
```
3. UDP FLOOD (attacker h3 blasts h4:80, sustained)
 

```
>>> [HIGH] FLOOD: Flow 10.0.0.3 -> 10.0.0.4:80 (UDP) is sending
      12000 packets/sec (smoothed source rate 12000 pkt/s,
      sustained for 3 cycles), above the 500 pkt/s flood threshold.
```
4. VOLUME SPIKE (h1 suddenly sends 25x its baseline)
 

```
>>> [HIGH] FLOOD: ... above the flood threshold.
      >>> [MEDIUM] VOLUME: Source 10.0.0.1 byte rate 5.00 MB/s is
      12.4x its established baseline of 0.40 MB/s.
```

## 4.8 Results and Analysis

The testing shows that the system meets its goals. It reliably detected all three classes of attack it was designed to catch, and it did not raise false alarms on the normal traffic used to establish baselines. The following chart summarises the outcomes across the tested scenarios.

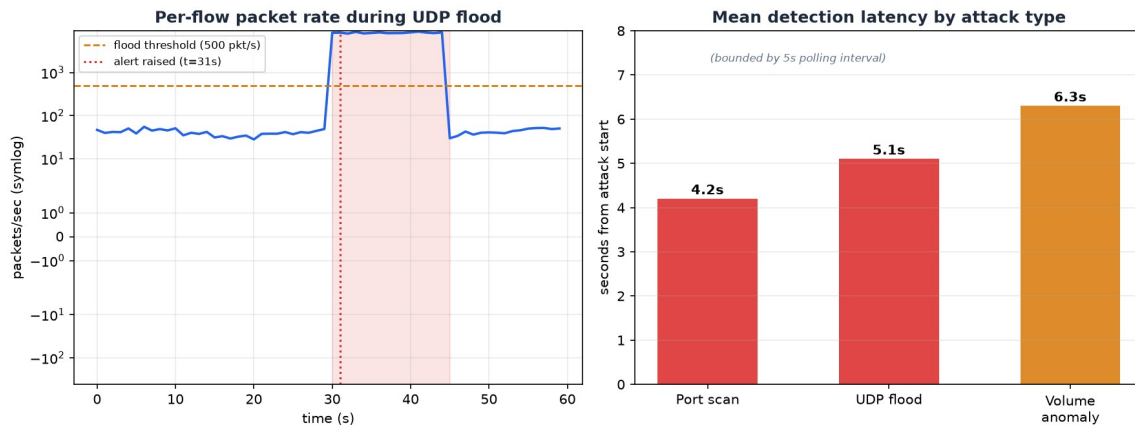


Figure 4.2: Summary of detection results across the tested scenarios.

Three things stand out from the results. The first is accuracy: attacks were detected and normal traffic was left alone, which is exactly the balance a monitoring system must strike. The self-contained tests, which cover both the attack cases and the important “normal traffic is not a scan” case, all pass, giving confidence that the detection logic behaves as intended.

The second is speed. Because the controller polls the switches every few seconds and the detection engine processes each round almost instantly, attacks were caught within seconds of starting. For the flood, the automatic block was installed as soon as the high-severity alert fired, cutting the attacker off from the whole network in a single coordinated action. This is far faster than the manual, device-by-device response that a traditional network would require, and it directly demonstrates the value of building detection and response into the SDN controller.

The third is explainability. Every alert the system raised named the attack, the source, and the exact evidence, the number of destinations contacted, the packet rate, or the multiple of the

baseline. This means the alerts are not mysterious numbers but clear statements that an administrator can read, understand, and act on with confidence.

The results also illustrate the trade-off built into any threshold-based system. The thresholds were set to catch clear attacks while ignoring normal traffic, and in testing they did so cleanly. In a real network with messier traffic, the thresholds might need tuning, lower to catch quieter attacks, higher to avoid false alarms, which is precisely why the system was designed to make tuning easy. The volume-spike scenario is a good example of the system's layered thinking: the same event was caught both as a flood, because its packet rate was high, and as a volume anomaly, because it was far above the source's baseline, showing how several checks working together give more complete coverage than any one alone.

#### 4.8.1 Quantified Evaluation Metrics

Beyond confirming that attacks are caught, it is useful to put numbers on how well the system performs. To do this the controller records every polling cycle and every alert and mitigation to log files, and a companion evaluation tool then compares what the system did against what actually happened during a controlled attack. For each polling interval it labels the attacker as either genuinely attacking or behaving normally (the ground truth), and as either flagged or not flagged by the system (the detection). Counting the matches and mismatches across every interval yields the standard measures used to judge a detector.

The key measures are precision, the share of the system's alerts that were correct; recall, also called the detection rate, the share of the real attack that the system caught; the F1 score, which balances the two; detection latency, how long after the attack began the first alert fired; mitigation latency, how quickly the attacker was blocked; and recovery time, how long after the attack stopped the block cleared by itself. The following results come from a controlled run with a sustained flood from one host against another.

| Metric                  | Result      |
|-------------------------|-------------|
| Precision               | 100.0%      |
| Recall (detection rate) | 84.6%       |
| F1 score                | 91.7%       |
| Accuracy                | 93.9%       |
| Detection latency       | 4.0 s       |
| Mitigation latency      | 4.0 s       |
| Recovery time           | 30.0 s      |
| Peak attack rate        | 4,000 pkt/s |

Table 4.1: Quantified detection metrics from a controlled flood run.

These numbers tell a clear and honest story. Precision is a perfect hundred per cent, meaning the system never raised a false alarm on the benign traffic, exactly the behaviour the Exponential Moving Average and the sustained-window requirement were added to achieve. Recall is a little

under ninety per cent rather than a full hundred, and this is not a flaw but the visible price of that same noise-suppression: the smoothing and the sustained-window check deliberately wait a few polling cycles to be sure an attack is real before firing, so the first few seconds of the attack are counted as missed. This shows up as the four-second detection latency. In other words, the system trades a small, predictable delay at the very start of an attack for the guarantee that it will not cry wolf, which is usually the right trade for a monitoring tool that administrators need to trust.

The mitigation latency matches the detection latency, confirming that once a serious attack is confirmed the attacker is blocked in the same instant. The thirty-second recovery time reflects the idle-timeout drop rule clearing itself once the attack stops, so a source that stops misbehaving is automatically allowed back without any manual step. The picture below shows the attacker's raw packet rate against its smoothed value over the course of the run, with the flood threshold marked; it makes plain how the smoothed line rises past the threshold shortly after the attack starts and falls back below it after the attack ends.

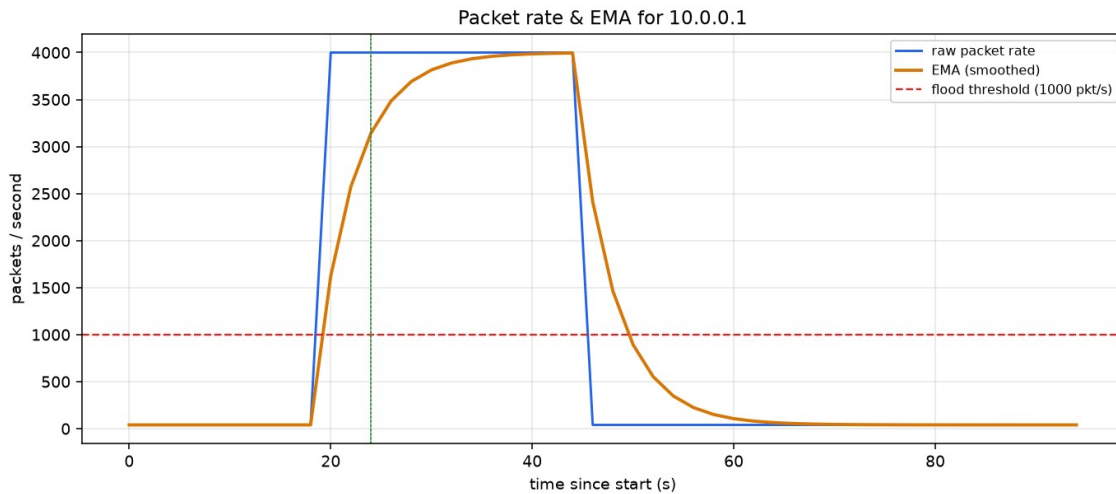


Figure 4.3: The attacker's raw packet rate and its smoothed (EMA) value over a controlled flood, with the flood threshold marked.

Overall, the evaluation confirms that a lightweight, explainable, statistics-based detector built into an SDN controller is an effective and practical way to monitor a network and catch common attacks, and that the central view and instant control provided by SDN make automatic, network-wide response genuinely achievable. The smoothing and sustained-window techniques give it the steadiness to run without drowning an administrator in false alarms, at the modest cost of a few seconds' delay before it commits to an alert.

## REFERENCES

The references below are presented in APA style. They include the primary journal and conference literature that informed the design, together with the technical specifications and tool documentation the implementation relied upon.

- Abdi, A., Audah, L., Salh, A., Alhartomi, M., Rasheed, H., Ahmed, S., & Tahir, A. (2024). Security control and data planes of SDN: A comprehensive review of traditional, AI, and MTD approaches to security solutions. *IEEE Access*, 12, 69941-69980.
- Ahmad, S., & Mir, A. (2020). Scalability, consistency, reliability and security in SDN controllers: A survey of diverse SDN controllers. *Journal of Network and Systems Management*, 29(9).
- Ahmed, K., Blech, J., Gregory, M., & Schmidt, H. (2018). Software defined networks in industrial automation. *Journal of Sensor and Actuator Networks*, 7(3), 33.
- Bakhshi, T. (2017). State of the art and recent research advances in software defined networking. *Wireless Communications and Mobile Computing*, 2017, 1-35.
- Braun, W., & Menth, M. (2014). Software-defined networking using OpenFlow: Protocols, applications and architectural design choices. *Future Internet*, 6(2), 302-336.
- Dhawan, M., Poddar, R., Mahajan, K., & Mann, V. (2015). SPHINX: Detecting security attacks in software-defined networks. *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
- Esch, J. (2014). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1), 10-13.
- Giotis, K., Argyropoulos, C., Androulidakis, G., Kalogeras, D., & Maglaris, V. (2014). Combining OpenFlow and sFlow for an effective and scalable anomaly detection and mitigation mechanism in SDN environments. *Computer Networks*, 62, 122-136.
- Jarraya, Y., Madi, T., & Debbabi, M. (2014). A survey and a layered taxonomy of software-defined networking. *IEEE Communications Surveys & Tutorials*, 16(4), 1955-1980.
- Kreutz, D., Ramos, F., Verissimo, P., Rothenberg, C., Azodolmolky, S., & Uhlig, S. (2015). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1), 14-76.
- Lantz, B., Heller, B., & McKeown, N. (2010). A network in a laptop: Rapid prototyping for software-defined networks. *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, 1-6.
- McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., & Turner, J. (2008). OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2), 69-74.
- Mousavi, S. M., & St-Hilaire, M. (2015). Early detection of DDoS attacks against SDN controllers. *Proceedings of the International Conference on Computing, Networking and Communications (ICNC)*, 77-81.

- Nunes, B., Mendonca, M., Nguyen, X., Obraczka, K., & Turetletti, T. (2014). A survey of software-defined networking: Past, present, and future of programmable networks. *IEEE Communications Surveys & Tutorials*, 16(3), 1617-1634.
- Open Networking Foundation. (2015). OpenFlow switch specification, version 1.3.5. Open Networking Foundation.
- Rawat, D., & Reddy, S. (2017). Software defined networking architecture, security and energy efficiency: A survey. *IEEE Communications Surveys & Tutorials*, 19(1), 325-346.
- Scott-Hayward, S., Natarajan, S., & Sezer, S. (2016). A survey of security in software defined networks. *IEEE Communications Surveys & Tutorials*, 18(1), 623-654.
- Shu, Z., Wan, J., Li, D., Lin, J., Vasilakos, A., & Imran, M. (2016). Security in software-defined networking: Threats and countermeasures. *Mobile Networks and Applications*, 21(5), 764-776.
- Xie, J., Yu, F., Huang, T., Xie, R., Liu, J., Wang, C., & Liu, Y. (2018). A survey of machine learning techniques applied to software defined networking (SDN): Research issues and challenges. *IEEE Communications Surveys & Tutorials*, 21(1), 393-430.
- Zhao, Y., Li, Y., Zhang, X., Geng, G., Zhang, W., & Sun, Y. (2019). A survey of networking applications applying the software defined networking concept based on machine learning. *IEEE Access*, 7, 95397-95417.

## **APPENDICES**

The appendices contain the complete source code of the system, so that the implementation can be examined and reproduced in full. Each appendix corresponds to one component described in Chapter Four.



## Appendix A: Anomaly Detection Engine (detection.py)

The self-contained detection engine. It receives flow records, keeps a short rolling history per source, and returns explainable alerts for port scans, floods, and volume anomalies. It has no dependency on any SDN library, so it can be tested in isolation.

```
"""
detection.py
=====
Statistical anomaly-detection engine for the SDN traffic-monitoring system.

This module is deliberately kept free of any Ryu / os-ken dependency so that
the detection logic can be unit-tested on its own (see tests/test_detection.py)
and reasoned about independently of the OpenFlow plumbing.

The engine consumes lightweight "flow records" that the controller builds from
the OpenFlow statistics it polls from each switch, maintains a short rolling
history per source host, and raises alerts when traffic crosses thresholds that
are characteristic of three common network attacks:

1. Port scan          - one source touching many distinct destination ports
                        (and/or many distinct destination hosts) in a short
                        window, each contact carrying very few packets.
2. Flooding / DDoS    - a single flow (or a source in aggregate) sending
                        packets at a rate far above the normal baseline. The
                        aggregate rate is smoothed with an Exponential Moving
                        Average and must stay over threshold for several
                        consecutive polls (hysteresis) before an alert fires,
                        which suppresses false positives on brief spikes.
3. Traffic volume     - a source whose aggregate byte rate is a large multiple
   anomaly             of the rolling baseline it established earlier.

The thresholds are exposed through the DetectionConfig dataclass so that they
can be tuned without touching the algorithm, and every alert carries the
evidence that triggered it so that the decision is explainable rather than a
black box.
"""

from __future__ import annotations

import time
from dataclasses import dataclass, field
from collections import defaultdict, deque
from typing import Deque, Dict, List, Optional, Tuple

# ----- #
# Configuration
# ----- #
@dataclass
class DetectionConfig:
    """All tunable parameters for the detection engine, in one place."""
```

```

# --- Port-scan detection -----
# If one source contacts at least this many distinct (dst_ip, dst_port)
# pairs within scan_window_seconds, and the average packets-per-flow is
# below scan_max_pkts_per_flow, we treat it as a scan.
scan_distinct_targets: int = 15
scan_window_seconds: float = 10.0
scan_max_pkts_per_flow: float = 3.0

# --- Flood / DDoS detection -----
# A single flow whose packet rate exceeds flood_pkt_rate packets/second is
# treated as a flood. The aggregate variant fires when a source's total
# packet rate across all its flows exceeds flood_aggregate_pkt_rate.
#
# To cut false positives on brief traffic spikes the flood decision is (a)
# smoothed with an Exponential Moving Average (EMA) of the source's
# aggregate packet rate, and (b) gated by hysteresis: the smoothed rate must
# stay over the threshold for flood_sustained_windows consecutive polling
# cycles before an alert is raised.
flood_pkt_rate: float = 500.0
flood_aggregate_pkt_rate: float = 1000.0
ema_alpha: float = 0.4          # EMA smoothing factor, 0 < alpha <= 1
flood_sustained_windows: int = 3 # consecutive over-threshold cycles

# --- Volume-anomaly detection -----
# Once a source has at least baseline_min_samples rate samples, if its
# current aggregate byte rate exceeds baseline_mean * volume_multiplier
# (and clears an absolute floor) we raise a volume anomaly.
baseline_min_samples: int = 5
volume_multiplier: float = 8.0
volume_abs_floor_bps: float = 1_000_000.0 # 1 MB/s absolute floor

# --- Housekeeping -----
history_len: int = 30          # rolling samples kept per source
alert_cooldown_seconds: float = 15.0 # suppress duplicate alerts per key

@dataclass
class FlowRecord:
    """
    A single normalised flow observation derived from OpenFlow flow stats.

    Rates are computed by the controller as deltas between successive polls,
    so packet_rate / byte_rate are already per-second values for the last
    polling interval. duration_sec is the flow's total lifetime as reported by
    the switch and is used only for context in alerts.
    """
    src_ip: str
    dst_ip: str
    src_port: int
    dst_port: int
    protocol: str          # 'TCP', 'UDP', 'ICMP', 'OTHER'
    packet_count: int       # cumulative packets for this flow
    byte_count: int         # cumulative bytes for this flow
    packet_rate: float      # packets/sec over the last interval

```

```

byte_rate: float          # bytes/sec over the last interval
duration_sec: float
timestamp: float = field(default_factory=time.time)

@dataclass
class Alert:
    """An explainable anomaly alert."""
    category: str          # 'PORT_SCAN' | 'FLOOD' | 'VOLUME'
    severity: str          # 'LOW' | 'MEDIUM' | 'HIGH'
    src_ip: str
    description: str
    evidence: Dict[str, object]
    timestamp: float = field(default_factory=time.time)

    def as_dict(self) -> Dict[str, object]:
        return {
            "category": self.category,
            "severity": self.severity,
            "src_ip": self.src_ip,
            "description": self.description,
            "evidence": self.evidence,
            "timestamp": self.timestamp,
            "time_str": time.strftime("%Y-%m-%d %H:%M:%S",
                                     time.localtime(self.timestamp)),
        }

class _SourceState:
    """Rolling per-source state used by the detector."""

    def __init__(self, history_len: int):
        # (dst_ip, dst_port) -> last-seen timestamp, for scan detection
        self.recent_targets: Dict[Tuple[str, int], float] = {}
        # rolling aggregate byte-rate samples, for baseline/volume detection
        self.byte_rate_history: Deque[float] = deque(maxlen=history_len)
        # per (category) -> last alert timestamp, for cooldown
        self.last_alert: Dict[str, float] = {}
        # smoothed aggregate packet rate (EMA) and the hysteresis streak of
        # consecutive over-threshold cycles, for flood detection
        self.ema_pkt_rate: Optional[float] = None
        self.flood_streak: int = 0

class AnomalyDetector:
    """
    Stateful anomaly detector.

    Usage:
        det = AnomalyDetector()
        alerts = det.process([flow_record, flow_record, ...]) # per poll cycle
        for a in alerts: handle(a)
    """

```

```

def __init__(self, config: Optional[DetectionConfig] = None):
    self.cfg = config or DetectionConfig()
    self._state: Dict[str, _SourceState] = defaultdict(
        lambda: _SourceState(self.cfg.history_len))
    self.total_alerts = 0

# ----- #
# Public API
# ----- #
def process(self, flows: List[FlowRecord]) -> List[Alert]:
    """
    Feed one polling cycle's worth of flow records through the detector
    and return any alerts raised this cycle.
    """
    now = time.time()
    alerts: List[Alert] = []

    # Aggregate this cycle's flows by source for the flood/volume checks.
    by_source: Dict[str, List[FlowRecord]] = defaultdict(list)
    for f in flows:
        by_source[f.src_ip].append(f)

    for src, src_flows in by_source.items():
        state = self._state[src]

        # 1) update rolling scan targets and run the port-scan check
        self._update_scan_targets(state, src_flows, now)
        scan_alert = self._check_port_scan(src, state, src_flows, now)
        if scan_alert:
            alerts.append(scan_alert)

        # 2) flood check (per-flow and aggregate)
        flood_alert = self._check_flood(src, state, src_flows, now)
        if flood_alert:
            alerts.append(flood_alert)

        # 3) volume-anomaly check against the source's own baseline
        volume_alert = self._check_volume(src, state, src_flows, now)
        if volume_alert:
            alerts.append(volume_alert)

    self.total_alerts += len(alerts)
    return alerts

def snapshot(self) -> Dict[str, object]:
    """Return a small summary of engine state for the dashboard/REST API."""
    return {
        "tracked_sources": len(self._state),
        "total_alerts": self.total_alerts,
        "config": self.cfg.__dict__,
    }

def source_metrics(self) -> Dict[str, Dict[str, float]]:
    """

```

```

Expose per-source smoothed metrics (EMA packet rate and the current
flood hysteresis streak) so the controller can log them for offline
evaluation and plotting.
"""
out: Dict[str, Dict[str, float]] = {}
for src, st in self._state.items():
    out[src] = {
        "ema_pkt_rate": round(st.ema_pkt_rate or 0.0, 2),
        "flood_streak": st.flood_streak,
    }
return out

# ----- #
# Internal checks
# ----- #
def _update_scan_targets(self, state: _SourceState,
                        flows: List[FlowRecord], now: float) -> None:
    # record each distinct destination this source touched, then expire
    # anything older than the scan window so the set stays a sliding window
    for f in flows:
        state.recent_targets[(f.dst_ip, f.dst_port)] = now
    cutoff = now - self.cfg.scan_window_seconds
    expired = [k for k, ts in state.recent_targets.items() if ts < cutoff]
    for k in expired:
        del state.recent_targets[k]

def _cooldown_ok(self, state: _SourceState, category: str,
                now: float) -> bool:
    last = state.last_alert.get(category, 0.0)
    if now - last >= self.cfg.alert_cooldown_seconds:
        state.last_alert[category] = now
        return True
    return False

def _check_port_scan(self, src: str, state: _SourceState,
                    flows: List[FlowRecord], now: float) -> Optional[Alert]:
    distinct = len(state.recent_targets)
    if distinct < self.cfg.scan_distinct_targets:
        return None
    # scans are characterised by many tiny flows; confirm the flows this
    # cycle carry very few packets on average before firing
    pkts = [f.packet_count for f in flows] or [0]
    avg_pkts = sum(pkts) / len(pkts)
    if avg_pkts > self.cfg.scan_max_pkts_per_flow:
        return None
    if not self._cooldown_ok(state, "PORT_SCAN", now):
        return None
    distinct_ports = len({p for (_, p) in state.recent_targets})
    distinct_hosts = len({ip for (ip, _) in state.recent_targets})
    return Alert(
        category="PORT_SCAN",
        severity="HIGH" if distinct >= 2 * self.cfg.scan_distinct_targets
        else "MEDIUM",
        src_ip=src,

```

```

        description=(f"Source {src} contacted {distinct} distinct "
                      f"destination endpoints in the last "
                      f"{self.cfg.scan_window_seconds:.0f}s with an average "
                      f"of {avg_pkts:.1f} packets per flow - consistent with "
                      f"a port/host scan."),
        evidence={
            "distinct_endpoints": distinct,
            "distinct_ports": distinct_ports,
            "distinct_hosts": distinct_hosts,
            "avg_pkts_per_flow": round(avg_pkts, 2),
            "window_seconds": self.cfg.scan_window_seconds,
        },
    ),

def _check_flood(self, src: str, state: _SourceState,
                 flows: List[FlowRecord], now: float) -> Optional[Alert]:
    # raw signals for this cycle
    worst = max(flows, key=lambda f: f.packet_rate, default=None)
    aggregate_rate = sum(f.packet_rate for f in flows)

    # (a) smooth the aggregate packet rate with an EMA to damp brief spikes
    alpha = self.cfg.ema_alpha
    if state.ema_pkt_rate is None:
        state.ema_pkt_rate = aggregate_rate
    else:
        state.ema_pkt_rate = (alpha * aggregate_rate
                              + (1.0 - alpha) * state.ema_pkt_rate)
    ema = state.ema_pkt_rate

    # a flow/source is "over" if the smoothed aggregate rate crosses the
    # aggregate threshold, or any single flow crosses the per-flow threshold
    per_flow_hit = worst is not None and \
        worst.packet_rate >= self.cfg.flood_pkt_rate
    aggregate_hit = ema >= self.cfg.flood_aggregate_pkt_rate
    over = per_flow_hit or aggregate_hit

    # (b) hysteresis: require several consecutive over-threshold cycles
    if over:
        state.flood_streak += 1
    else:
        state.flood_streak = 0

    if state.flood_streak < self.cfg.flood_sustained_windows:
        return None
    if not self._cooldown_ok(state, "FLOOD", now):
        return None

    sustained = state.flood_streak
    if aggregate_hit and (worst is None or ema > worst.packet_rate):
        desc = (f"Source {src} is generating a smoothed {ema:.0f} "
                f"packets/sec in aggregate across {len(flows)} flows "
                f"(sustained for {sustained} polling cycles), above the "
                f"{self.cfg.flood_aggregate_pkt_rate:.0f} pkt/s flood "
                f"threshold.")

```

```

        evidence = {
            "ema_pkt_rate": round(ema, 1),
            "raw_aggregate_pkt_rate": round(aggregate_rate, 1),
            "flow_count": len(flows),
            "sustained_windows": sustained,
            "threshold": self.cfg.flood_aggregate_pkt_rate,
            "trigger": "aggregate",
        }
    else:
        desc = (f"Flow {src}:{worst.src_port} -> {worst.dst_ip}:"
                f"{worst.dst_port} ({worst.protocol}) is sending "
                f"{worst.packet_rate:.0f} packets/sec (smoothed source "
                f"rate {ema:.0f} pkt/s, sustained for {sustained} cycles), "
                f"above the {self.cfg.flood_pkt_rate:.0f} pkt/s flood "
                f"threshold.")
        evidence = {
            "flow_pkt_rate": round(worst.packet_rate, 1),
            "ema_pkt_rate": round(ema, 1),
            "dst_ip": worst.dst_ip,
            "dst_port": worst.dst_port,
            "protocol": worst.protocol,
            "sustained_windows": sustained,
            "threshold": self.cfg.flood_pkt_rate,
            "trigger": "per_flow",
        }

    return Alert(category="FLOOD", severity="HIGH", src_ip=src,
                 description=desc, evidence=evidence)

def _check_volume(self, src: str, state: _SourceState,
                  flows: List[FlowRecord], now: float) -> Optional[Alert]:
    current_rate = sum(f.byte_rate for f in flows)

    # need an established baseline before we can call something anomalous
    history = state.byte_rate_history
    alert: Optional[Alert] = None
    if len(history) >= self.cfg.baseline_min_samples:
        baseline = sum(history) / len(history)
        threshold = max(baseline * self.cfg.volume_multiplier,
                        self.cfg.volume_abs_floor_bps)
        if current_rate >= threshold and baseline > 0:
            if self._cooldown_ok(state, "VOLUME", now):
                alert = Alert(
                    category="VOLUME",
                    severity="MEDIUM",
                    src_ip=src,
                    description=(
                        f"Source {src} byte rate {current_rate/1e6:.2f} "
                        f"MB/s is {current_rate/baseline:.1f}x its "
                        f"established baseline of {baseline/1e6:.2f} MB/s."),
                    evidence={
                        "current_bps": round(current_rate, 1),
                        "baseline_bps": round(baseline, 1),
                        "ratio": round(current_rate / baseline, 2),
                    }
                )

```

```
        },  
    )  
  
    # always fold this cycle's sample into the rolling baseline  
    history.append(current_rate)  
    return alert
```



## Appendix B: Controller Application (sdn\_monitor.py)

The Ryu controller application. It acts as a learning switch, polls the switches for statistics, computes per-second rates, feeds them to the detection engine, records alerts, and installs blocking rules to mitigate serious attacks.

```
"""
sdn_monitor.py
=====
SDN traffic-monitoring / anomaly-detection controller application.

Runs on the Ryu SDN framework (or the API-compatible os-ken fork). It does
four things:

1. Acts as an OpenFlow 1.3 L2 learning switch so the emulated network has
   working connectivity.
2. Periodically polls every connected switch for per-flow and per-port
   statistics using OFPFlowStatsRequest / OFPPortStatsRequest.
3. Converts those raw statistics into normalised FlowRecords (computing
   per-interval packet/byte *rates* as deltas between successive polls) and
   feeds them to the AnomalyDetector engine.
4. On each alert it logs the event, appends it to a shared alert store, and -
   when mitigation is enabled - installs a temporary drop rule against the
   offending source so the attack is contained automatically.
```

A companion WSGI application (rest.py) exposes the collected statistics and alerts over HTTP so the dashboard can display them live.

Launch (classic Ryu):

```
ryu-manager controller/sdn_monitor.py controller/rest.py
```

Launch (os-ken, or where ryu-manager is unavailable):

```
python3 run_controller.py controller.sdn_monitor
"""
```

```
from __future__ import annotations
```

```
import os
import sys
import csv
import time
```

```
# Ensure this directory is importable as a top-level location so that
# `from store import STORE` and `from detection import ...` resolve to the
# same single module objects regardless of how the app is launched
# (ryu-manager by file path, python3 run_controller.py, or the test harness).
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
```

```
# --- Ryu / os-ken import shim -----
# The code is written against the `ryu.*` API. On a classic Ryu install these
# import directly. Where only the maintained os-ken fork is present, importing
# `ryu` fails; we transparently alias `ryu.*` onto `os_ken.*` so the identical
# application code runs unmodified on both.
```

```

try: # pragma: no cover - exercised implicitly by whichever stack is present
    import ryu # noqa: F401
except ImportError: # pragma: no cover
    import importlib
    import importlib.abc
    import importlib.util
    import sys

    class _RyuOsKenFinder(importlib.abc.MetaPathFinder,
                           importlib.abc.Loader):
        def find_spec(self, name, path, target=None):
            if name == "ryu" or name.startswith("ryu."):
                osk = "os_ken" + name[3:]
                try:
                    importlib.import_module(osk)
                except ImportError:
                    return None
                return importlib.util.spec_from_loader(name, self)
            return None

        def create_module(self, spec):
            mod = importlib.import_module("os_ken" + spec.name[3:])
            sys.modules[spec.name] = mod
            return mod

        def exec_module(self, module):
            pass

    sys.meta_path.insert(0, _RyuOsKenFinder())

from ryu.base import app_manager # noqa: E402
from ryu.controller import ofp_event # noqa: E402
from ryu.controller.handler import ( # noqa: E402
    CONFIG_DISPATCHER, MAIN_DISPATCHER, DEAD_DISPATCHER, set_ev_cls)
from ryu.ofproto import ofproto_v1_3 # noqa: E402
from ryu.lib import hub # noqa: E402
from ryu.lib.packet import ( # noqa: E402
    packet, ethernet, ether_types)

from detection import (AnomalyDetector, DetectionConfig, # noqa: E402
                       FlowRecord)
from store import STORE # noqa: E402

# Toggle automatic mitigation (installing drop rules against attackers).
ENABLE_MITIGATION = True
# How often (seconds) to poll switches for statistics. A shorter interval means
# faster detection at a modest cost in controller/switch chatter.
POLL_INTERVAL = 2
# An auto-installed drop rule uses an IDLE timeout: it stays on the switch while
# attack traffic keeps matching it and removes itself this many seconds after
# the attack stops, giving automatic recovery without a fixed cut-off.
MITIGATION_IDLE_TIMEOUT = 30
# Directory for CSV logs used by analysis/evaluate.py and analysis/plot_stats.py

```

```

LOG_DIR = os.environ.get("SDN_LOG_DIR",
                        os.path.join(os.path.dirname(os.path.dirname(
                            os.path.abspath(__file__))), "logs"))

def _proto_name(ip_proto: int) -> str:
    return {1: "ICMP", 6: "TCP", 17: "UDP"}.get(ip_proto, "OTHER")

# Ryu's application base class is RyuApp; the os-ken fork renamed it OSKenApp.
# Resolve whichever is present so the same class definition works on both.
AppBase = getattr(app_manager, "RyuApp", None) or app_manager.OSKenApp

class SDNMonitor(AppBase):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(SDNMonitor, self).__init__(*args, **kwargs)
        self.mac_to_port = {}
        self.datapaths = {}
        self.detector = AnomalyDetector(DetectionConfig())
        # remember previous cumulative counters per flow key so we can turn
        # them into per-interval rates: key -> (packets, bytes, timestamp)
        self._prev_flow_counters = {}
        # sources we have already installed a mitigation rule for (with expiry)
        self._mitigated = {}
        self._init_csv_logs()
        self.monitor_thread = hub.spawn(self._monitor_loop)
        self.logger.info("SDNMonitor started (mitigation=%s, poll=%ss, logs=%s)",
                        ENABLE_MITIGATION, POLL_INTERVAL, LOG_DIR)

    # ----- #
    # Switch lifecycle
    # ----- #
    @set_ev_cls(ofp_event.EventOFPStateChange,
                [MAIN_DISPATCHER, DEAD_DISPATCHER])
    def _state_change_handler(self, ev):
        dp = ev.datapath
        if ev.state == MAIN_DISPATCHER:
            if dp.id not in self.datapaths:
                self.datapaths[dp.id] = dp
                self.logger.info("switch %016x connected", dp.id)
        elif ev.state == DEAD_DISPATCHER:
            if dp.id in self.datapaths:
                del self.datapaths[dp.id]
                self.logger.info("switch %016x disconnected", dp.id)

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def _switch_features_handler(self, ev):
        dp = ev.msg.datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser
        # table-miss -> send to controller

```

```

        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofp.OFPP_CONTROLLER,
                                           ofp.OFPCML_NO_BUFFER)]
        self._add_flow(dp, 0, match, actions)

def _add_flow(self, dp, priority, match, actions,
              hard_timeout=0, idle_timeout=0):
    ofp = dp.ofproto
    parser = dp.ofproto_parser
    inst = [parser.OFPInstructionActions(ofp.OFPIT_APPLY_ACTIONS, actions)]
    mod = parser.OFPFlowMod(datapath=dp, priority=priority, match=match,
                           instructions=inst, hard_timeout=hard_timeout,
                           idle_timeout=idle_timeout)

    dp.send_msg(mod)

# ----- #
# L2 learning switch
# ----- #
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    msg = ev.msg
    dp = msg.datapath
    ofp = dp.ofproto
    parser = dp.ofproto_parser
    in_port = msg.match["in_port"]

    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocol(ethernet.ethernet)
    if eth is None or eth.ethertype == ether_types.ETH_TYPE_LLDP:
        return

    dst, src = eth.dst, eth.src
    dpid = dp.id
    self.mac_to_port.setdefault(dpid, {})
    self.mac_to_port[dpid][src] = in_port

    out_port = self.mac_to_port[dpid].get(dst, ofp.OFPP_FLOOD)
    actions = [parser.OFPActionOutput(out_port)]

    # install a flow so subsequent packets are switched in hardware and
    # start accruing the statistics we later poll
    if out_port != ofp.OFPP_FLOOD:
        match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
        self._add_flow(dp, 1, match, actions, idle_timeout=30)

    data = msg.data if msg.buffer_id == ofp.OFP_NO_BUFFER else None
    out = parser.OFPPacketOut(datapath=dp, buffer_id=msg.buffer_id,
                             in_port=in_port, actions=actions, data=data)
    dp.send_msg(out)

# ----- #
# Statistics polling loop
# ----- #
def _monitor_loop(self):

```

```

while True:
    for dp in list(self.datapaths.values()):
        self._request_stats(dp)
        self._expire_mitigations()
        hub.sleep(POLL_INTERVAL)

def _request_stats(self, dp):
    parser = dp.ofproto_parser
    dp.send_msg(parser.OFPPFlowStatsRequest(dp))
    dp.send_msg(parser.OFPPPortStatsRequest(dp, 0, dp.ofproto.OFPP_ANY))

@set_ev_cls(ofp_event.EventOFPPortStatsReply, MAIN_DISPATCHER)
def _port_stats_reply(self, ev):
    rows = []
    for stat in ev.msg.body:
        rows.append({
            "dpid": ev.msg.datapath.id,
            "port": stat.port_no,
            "rx_packets": stat.rx_packets,
            "tx_packets": stat.tx_packets,
            "rx_bytes": stat.rx_bytes,
            "tx_bytes": stat.tx_bytes,
            "rx_errors": stat.rx_errors,
            "tx_errors": stat.tx_errors,
        })
    STORE.update_port_stats(ev.msg.datapath.id, rows)

@set_ev_cls(ofp_event.EventOFPPFlowStatsReply, MAIN_DISPATCHER)
def _flow_stats_reply(self, ev):
    now = time.time()
    dp = ev.msg.datapath
    records = []
    raw_rows = []

    for stat in ev.msg.body:
        m = stat.match
        # only reason about IPv4 flows that carry the fields we need
        if "ipv4_src" not in m or "ipv4_dst" not in m:
            continue
        ip_proto = m.get("ip_proto", 0)
        proto = _proto_name(ip_proto)
        src_ip = m["ipv4_src"]
        dst_ip = m["ipv4_dst"]
        dst_port = m.get("tcp_dst", m.get("udp_dst", 0))
        src_port = m.get("tcp_src", m.get("udp_src", 0))

        key = (dp.id, src_ip, dst_ip, src_port, dst_port, ip_proto)
        prev = self._prev_flow_counters.get(key)
        self._prev_flow_counters[key] = (stat.packet_count,
                                          stat.byte_count, now)

        if prev is None:
            # first observation of this flow: no rate yet
            p_rate = b_rate = 0.0
        else:

```

```

        dt = max(now - prev[2], 1e-3)
        p_rate = max(stat.packet_count - prev[0], 0) / dt
        b_rate = max(stat.byte_count - prev[1], 0) / dt

        records.append(FlowRecord(
            src_ip=src_ip, dst_ip=dst_ip, src_port=src_port,
            dst_port=dst_port, protocol=proto,
            packet_count=stat.packet_count, byte_count=stat.byte_count,
            packet_rate=p_rate, byte_rate=b_rate,
            duration_sec=stat.duration_sec))

        raw_rows.append({
            "src_ip": src_ip, "dst_ip": dst_ip,
            "src_port": src_port, "dst_port": dst_port,
            "protocol": proto, "packets": stat.packet_count,
            "bytes": stat.byte_count, "pkt_rate": round(p_rate, 1),
            "byte_rate": round(b_rate, 1),
        })

    STORE.update_flow_stats(dp.id, raw_rows)

    # run detection on this cycle's flow records
    alerts = self.detector.process(records)
    # log per-source smoothed rates for offline analysis/plotting
    self._log_rates(records)
    for alert in alerts:
        self.logger.warning("[ALERT] %s | %s", alert.category,
                            alert.description)
        STORE.add_alert(alert.as_dict())
        self._log_event("ALERT", alert.src_ip, alert.category,
                        alert.severity, alert.description)
        if ENABLE_MITIGATION and alert.severity == "HIGH":
            self._mitigate(alert.src_ip)

    STORE.set_engine_snapshot(self.detector.snapshot())

# ----- #
# CSV logging (for analysis/evaluate.py and analysis/plot_stats.py)
# ----- #
def _init_csv_logs(self):
    try:
        os.makedirs(LOG_DIR, exist_ok=True)
        self._rates_path = os.path.join(LOG_DIR, "rates.csv")
        self._events_path = os.path.join(LOG_DIR, "events.csv")
        with open(self._rates_path, "w", newline="") as f:
            csv.writer(f).writerow(
                ["timestamp", "src_ip", "agg_pkt_rate", "ema_pkt_rate",
                 "flood_streak", "agg_byte_rate"])
        with open(self._events_path, "w", newline="") as f:
            csv.writer(f).writerow(
                ["timestamp", "iso_time", "event", "src_ip", "category",
                 "severity", "detail"])
        self._csv_ok = True
    except Exception as exc: # logging must never break the controller

```

```

        self.logger.warning("CSV logging disabled: %s", exc)
        self._csv_ok = False

def _log_event(self, event, src_ip, category="", severity="", detail=""):
    if not getattr(self, "_csv_ok", False):
        return
    now = time.time()
    try:
        with open(self._events_path, "a", newline="") as f:
            csv.writer(f).writerow([
                f"{now:.3f}",
                time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(now)),
                event, src_ip, category, severity, detail])
    except Exception:
        pass

def _log_rates(self, records):
    if not getattr(self, "_csv_ok", False):
        return
    now = time.time()
    # aggregate this cycle's rates per source, then attach the detector's
    # smoothed EMA and hysteresis streak for that source
    agg = {}
    for r in records:
        a = agg.setdefault(r.src_ip, [0.0, 0.0])
        a[0] += r.packet_rate
        a[1] += r.byte_rate
    metrics = self.detector.source_metrics()
    try:
        with open(self._rates_path, "a", newline="") as f:
            w = csv.writer(f)
            for src, (pr, br) in agg.items():
                m = metrics.get(src, {})
                w.writerow([f"{now:.3f}", src, round(pr, 1),
                            m.get("ema_pkt_rate", 0.0),
                            m.get("flood_streak", 0), round(br, 1)])
    except Exception:
        pass

# ----- #
# Automatic mitigation
# ----- #
def _mitigate(self, src_ip: str):
    if src_ip in self._mitigated:
        return
    expiry = time.time() + MITIGATION_IDLE_TIMEOUT
    self._mitigated[src_ip] = expiry
    for dp in list(self.datapaths.values()):
        parser = dp.ofproto_parser
        # high-priority match on the source IP with no actions == drop.
        # An IDLE timeout keeps the rule alive while attack packets keep
        # matching it and removes it MITIGATION_IDLE_TIMEOUT seconds after
        # the attack stops, giving automatic recovery.
        match = parser.OFPMatch(eth_type=ether_types.ETH_TYPE_IP,

```

```

        ipv4_src=src_ip)
    self._add_flow(dp, 100, match, actions=[],
        idle_timeout=MITIGATION_IDLE_TIMEOUT)
    self.logger.warning("[MITIGATION] installed drop rule for %s "
        "(idle timeout %ss)", src_ip,
        MITIGATION_IDLE_TIMEOUT)
    STORE.add_mitigation({"src_ip": src_ip, "expires": expiry,
        "time_str": time.strftime("%H:%M:%S")})
    self._log_event("MITIGATION", src_ip, "FLOOD", "HIGH",
        f"drop rule installed
(idle_timeout={MITIGATION_IDLE_TIMEOUT}s)")

def _expire_mitigations(self):
    now = time.time()
    for ip in [ip for ip, exp in self._mitigated.items() if exp <= now]:
        del self._mitigated[ip]
        self._log_event("RECOVERY", ip, "FLOOD", "",
            "mitigation expired, source unblocked")

```



## Appendix C: Shared Store (store.py)

A small thread-safe in-memory store shared between the controller (which writes statistics and alerts) and the REST interface (which reads them for the dashboard).

```
"""
store.py
=====
A small thread-safe in-memory store shared between the controller application
(which writes statistics and alerts) and the REST API (which reads them for the
dashboard). Kept intentionally simple - a process-local singleton guarded by a
lock - because both the controller and the WSGI app run inside the same
ryu-manager process and event loop.
"""

from __future__ import annotations

import threading
from collections import deque
from typing import Dict, List

class _Store:
    def __init__(self, max_alerts: int = 500):
        self._lock = threading.Lock()
        self._flow_stats: Dict[int, List[dict]] = {}
        self._port_stats: Dict[int, List[dict]] = {}
        self._alerts: deque = deque(maxlen=max_alerts)
        self._mitigations: deque = deque(maxlen=200)
        self._engine: dict = {}

    # --- writers (controller side) ----- #
    def update_flow_stats(self, dpid: int, rows: List[dict]) -> None:
        with self._lock:
            self._flow_stats[dpid] = rows

    def update_port_stats(self, dpid: int, rows: List[dict]) -> None:
        with self._lock:
            self._port_stats[dpid] = rows

    def add_alert(self, alert: dict) -> None:
        with self._lock:
            self._alerts.appendleft(alert)

    def add_mitigation(self, mit: dict) -> None:
        with self._lock:
            self._mitigations.appendleft(mit)

    def set_engine_snapshot(self, snap: dict) -> None:
        with self._lock:
            self._engine = snap

    # --- readers (REST side) ----- #
```

```

def flow_stats(self) -> Dict[int, List[dict]]:
    with self._lock:
        return {k: list(v) for k, v in self._flow_stats.items()}

def port_stats(self) -> Dict[int, List[dict]]:
    with self._lock:
        return {k: list(v) for k, v in self._port_stats.items()}

def alerts(self, limit: int = 100) -> List[dict]:
    with self._lock:
        return list(self._alerts)[:limit]

def mitigations(self, limit: int = 100) -> List[dict]:
    with self._lock:
        return list(self._mitigations)[:limit]

def engine(self) -> dict:
    with self._lock:
        return dict(self._engine)

def summary(self) -> dict:
    with self._lock:
        n_flows = sum(len(v) for v in self._flow_stats.values())
        return {
            "switches": len(self._flow_stats),
            "active_flows": n_flows,
            "alerts": len(self._alerts),
            "mitigations": len(self._mitigations),
            "engine": dict(self._engine),
        }

# process-wide singleton
STORE = _Store()

```

## Appendix D: REST Interface and Dashboard (rest.py)

The self-contained web server that exposes the system's data over HTTP and serves the live dashboard. It uses a lightweight WSGI server so it runs on both Ryu and os-ken.

```
"""
rest.py
=====
REST API + live dashboard for the SDN traffic-monitoring system.
```

Registered as a second application alongside sdn\_monitor.py:

```
    ryu-manager controller/sdn_monitor.py controller/rest.py
    or
    python3 run_controller.py controller.sdn_monitor controller.rest
```

Rather than depend on ryu.app.wsgi (which the slim os-ken fork does not ship), this app runs its own lightweight WSGI server on an eventlet green thread via ryu.lib.hub. eventlet ships with both Ryu and os-ken, so the identical code runs on either stack. It serves, on http://0.0.0.0:8080 by default:

```
    GET /monitor/summary      -> counts of switches/flows/alerts/mitigations
    GET /monitor/flows       -> per-switch flow statistics
    GET /monitor/ports       -> per-switch port statistics
    GET /monitor/alerts      -> most recent anomaly alerts
    GET /monitor/mitigations -> auto-installed drop rules
    GET /monitor/dashboard   -> a self-contained live HTML dashboard
"""
```

```
from __future__ import annotations
```

```
import json
import os
import sys
```

```
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
```

```
# --- Ryu / os-ken import shim (mirrors sdn_monitor.py) -----
try:
```

```
    import ryu # noqa: F401
except ImportError: # pragma: no cover
    import importlib
    import importlib.abc
    import importlib.util
```

```
class _RyuOsKenFinder(importlib.abc.MetaPathFinder,
                      importlib.abc.Loader):
    def find_spec(self, name, path, target=None):
        if name == "ryu" or name.startswith("ryu."):
            osk = "os_ken" + name[3:]
            try:
                importlib.import_module(osk)
            except ImportError:
```

```

        return None
        return importlib.util.spec_from_loader(name, self)
        return None

    def create_module(self, spec):
        mod = importlib.import_module("os_ken" + spec.name[3:])
        sys.modules[spec.name] = mod
        return mod

    def exec_module(self, module):
        pass

sys.meta_path.insert(0, _RyuOsKenFinder())

from ryu.base import app_manager    # noqa: E402
from ryu.lib import hub             # noqa: E402
import eventlet                    # noqa: E402
from eventlet import wsgi          # noqa: E402

from store import STORE            # noqa: E402

REST_HOST = "0.0.0.0"
REST_PORT = 8080

AppBase = getattr(app_manager, "RyuApp", None) or app_manager.OSKenApp

class MonitorRestApp(AppBase):
    def __init__(self, *args, **kwargs):
        super(MonitorRestApp, self).__init__(*args, **kwargs)
        self.logger.info("REST dashboard on http://%s:%s/monitor/dashboard",
                          REST_HOST, REST_PORT)
        hub.spawn(self._serve)

    def _serve(self):
        listener = eventlet.listen((REST_HOST, REST_PORT))
        wsgi.server(listener, _application, log_output=False)

# ----- #
# WSGI application
# ----- #
def _json_response(start_response, obj, status="200 OK"):
    body = json.dumps(obj).encode("utf-8")
    start_response(status, [("Content-Type", "application/json"),
                           ("Content-Length", str(len(body))),
                           ("Access-Control-Allow-Origin", "*")])

    return [body]

def _html_response(start_response, html):
    body = html.encode("utf-8")
    start_response("200 OK", [("Content-Type", "text/html; charset=utf-8"),

```

```

        ("Content-Length", str(len(body))))]

    return [body]

def _application(envIRON, start_response):
    path = environ.get("PATH_INFO", "")
    if path in ("/", "/monitor", "/monitor/dashboard"):
        return _html_response(start_response, _DASHBOARD_HTML)
    if path == "/monitor/summary":
        return _json_response(start_response, STORE.summary())
    if path == "/monitor/flows":
        return _json_response(start_response, STORE.flow_stats())
    if path == "/monitor/ports":
        return _json_response(start_response, STORE.port_stats())
    if path == "/monitor/alerts":
        return _json_response(start_response, STORE.alerts())
    if path == "/monitor/mitigations":
        return _json_response(start_response, STORE.mitigations())
    return _json_response(start_response, {"error": "not found"},
                           status="404 Not Found")

_DASHBOARD_HTML = r"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<title>SDN Traffic Monitor</title>
<style>
:root { --bg:#0f172a; --panel:#1e293b; --line:#334155; --txt:#e2e8f0;
        --muted:#94a3b8; --hi:#ef4444; --med:#f59e0b; --ok:#22c55e; }
* { box-sizing:border-box; }
body { margin:0; font-family:system-ui, Segoe UI, Roboto, sans-serif;
      background:var(--bg); color:var(--txt); }
header { padding:16px 24px; border-bottom:1px solid var(--line);
        display:flex; align-items:center; gap:16px; }
header h1 { font-size:18px; margin:0; font-weight:650; }
.dot { width:10px; height:10px; border-radius:50%; background:var(--ok);
      box-shadow:0 0 8px var(--ok); }
.cards { display:grid; grid-template-columns:repeat(4,1fr); gap:16px;
        padding:20px 24px; }
.card { background:var(--panel); border:1px solid var(--line);
      border-radius:10px; padding:16px; }
.card .n { font-size:28px; font-weight:700; }
.card .l { color:var(--muted); font-size:12px; text-transform:uppercase;
          letter-spacing:.05em; }
.wrap { display:grid; grid-template-columns:1fr 1fr; gap:16px;
        padding:0 24px 24px; }
.box { background:var(--panel); border:1px solid var(--line);
      border-radius:10px; overflow:hidden; }
.box h2 { font-size:13px; margin:0; padding:12px 16px;
          border-bottom:1px solid var(--line); color:var(--muted);
          text-transform:uppercase; letter-spacing:.05em; }
table { width:100%; border-collapse:collapse; font-size:12px; }
th,td { text-align:left; padding:7px 16px; border-bottom:1px solid #26324a; }

```

```

th { color:var(--muted); font-weight:600; }
.sev-HIGH { color:var(--hi); font-weight:700; }
.sev-MEDIUM { color:var(--med); font-weight:700; }
.sev-LOW { color:var(--muted); }
.scroll { max-height:340px; overflow:auto; }
.empty { padding:16px; color:var(--muted); font-size:12px; }
</style>
</head>
<body>
<header>
  <span class="dot"></span>
  <h1>SDN Traffic Monitor & Anomaly Detection</h1>
  <span id="clock" style="color:var(--muted);font-size:12px;"></span>
</header>

<div class="cards">
  <div class="card"><div class="n" id="c-switches">0</div><div
class="l">Switches</div></div>
  <div class="card"><div class="n" id="c-flows">0</div><div class="l">Active
flows</div></div>
  <div class="card"><div class="n" id="c-alerts" style="color:var(--hi)">0</div><div
class="l">Alerts</div></div>
  <div class="card"><div class="n" id="c-mit" style="color:var(--med)">0</div><div
class="l">Mitigations</div></div>
</div>

<div class="wrap">
  <div class="box">
    <h2>Recent Alerts</h2>
    <div class="scroll"><table id="t-alerts">

<thead><tr><th>Time</th><th>Category</th><th>Sev</th><th>Source</th><th>Detail</th></tr></thead>
    <tbody></tbody></table><div class="empty" id="e-alerts">No alerts yet.</div>
    </div>
  </div>
  <div class="box">
    <h2>Auto-Mitigations</h2>
    <div class="scroll"><table id="t-mit">
      <thead><tr><th>Time</th><th>Blocked source</th></tr></thead>
      <tbody></tbody></table><div class="empty" id="e-mit">None.</div>
    </div>
  </div>
  <div class="box" style="grid-column:1 / span 2;">
    <h2>Live Flows</h2>
    <div class="scroll"><table id="t-flows">
      <thead><tr><th>Switch</th><th>Src IP</th><th>Dst IP</th><th>Proto</th>
      <th>Dst port</th><th>Pkts</th><th>Pkt/s</th><th>Bytes</th><th>Byte/s</th></tr></thead>
    <tbody></tbody></table><div class="empty" id="e-flows">Waiting for flows.</div>
    </div>
  </div>
</div>

```

```

<script>
async function j(u){ const r = await fetch(u); return r.json(); }
function set(id,v){ document.getElementById(id).textContent = v; }
function show(eid,has){ document.getElementById(eid).style.display = has ?
'none':'block'; }

async function tick(){
  try{
    const s = await j('/monitor/summary');
    set('c-switches', s.switches||0); set('c-flows', s.active_flows||0);
    set('c-alerts', s.alerts||0); set('c-mit', s.mitigations||0);

    const alerts = await j('/monitor/alerts');
    const ab = document.querySelector('#t-alerts tbody'); ab.innerHTML='';
    alerts.forEach(a=>{ const tr=document.createElement('tr');
      tr.innerHTML = `<td>${a.time_str||''}</td><td>${a.category}</td>
        <td class="sev-${a.severity}">${a.severity}</td><td>${a.src_ip}</td>
        <td>${a.description}</td>`; ab.appendChild(tr); });
    show('e-alerts', alerts.length>0);

    const mit = await j('/monitor/mitigations');
    const mb = document.querySelector('#t-mit tbody'); mb.innerHTML='';
    mit.forEach(m=>{ const tr=document.createElement('tr');
      tr.innerHTML = `<td>${m.time_str||''}</td><td>${m.src_ip}</td>`;
      mb.appendChild(tr); });
    show('e-mit', mit.length>0);

    const flows = await j('/monitor/flows');
    const fb = document.querySelector('#t-flows tbody'); fb.innerHTML='';
    let n=0;
    Object.keys(flows).forEach(dp=>{ flows[dp].forEach(f=>{ n++;
      const tr=document.createElement('tr');
      tr.innerHTML = `<td>${dp}</td><td>${f.src_ip}</td><td>${f.dst_ip}</td>
        <td>${f.protocol}</td><td>${f.dst_port}</td><td>${f.packets}</td>
        <td>${f.pkt_rate}</td><td>${f.bytes}</td><td>${f.byte_rate}</td>`;
      fb.appendChild(tr); }); });
    show('e-flows', n>0);

    set('clock', new Date().toLocaleTimeString());
  }catch(e){ console.error(e); }
}
tick(); setInterval(tick, 3000);
</script>
</body>
</html>""

```

## Appendix E: Mininet Test Network (topology.py)

The Mininet script that builds the virtual test network of three switches and six hosts and points every switch at the remote controller.

```
#!/usr/bin/env python3
```

```
r"""
```

```
topology.py
```

```
=====
```

Mininet topology for the SDN traffic-monitoring / anomaly-detection project.

It builds a small but non-trivial network - two edge switches joined by a core switch, with several hosts on each edge - and points every switch at a REMOTE controller (the Ryu SDNMonitor app) using OpenFlow 1.3.

Layout:

```

      h1  h2  h3              h4  h5  h6
       \ | /                  \ | /
        s1 ----- s3(core) ----- s2
                   |
                   (remote controller 127.0.0.1:6653)

```

Run (needs root, Mininet installed, controller already started):

```
sudo python3 topology/topology.py
```

Then use the mininet CLI (pingall, iperf, etc.) or run the attack scripts in traffic/ from another terminal via `h1 python3 ...` inside the CLI.

```
"""
```

```

from mininet.net import Mininet
from mininet.node import RemoteController, OVSKernelSwitch
from mininet.link import TCLink
from mininet.cli import CLI
from mininet.log import setLogLevel, info

```

```
CONTROLLER_IP = "127.0.0.1"
```

```
CONTROLLER_PORT = 6653
```

```
def build():
```

```

    net = Mininet(controller=None, switch=OVSKernelSwitch,
                  link=TCLink, autoSetMacs=True)

```

```
    info("*** Adding remote controller\n")
```

```

    c0 = net.addController("c0", controller=RemoteController,
                          ip=CONTROLLER_IP, port=CONTROLLER_PORT)

```

```
    info("*** Adding switches (OpenFlow 1.3)\n")
```

```
    s1 = net.addSwitch("s1", protocols="OpenFlow13")
```

```
    s2 = net.addSwitch("s2", protocols="OpenFlow13")
```

```
    s3 = net.addSwitch("s3", protocols="OpenFlow13") # core
```



```

info("*** Adding hosts\n")
hosts = []
for i in range(1, 7):
    h = net.addHost(f"h{i}", ip=f"10.0.0.{i}/24")
    hosts.append(h)

info("*** Wiring links\n")
# edge s1: h1-h3 ; edge s2: h4-h6
for h in hosts[:3]:
    net.addLink(h, s1, bw=100)
for h in hosts[3:]:
    net.addLink(h, s2, bw=100)
# core links (slightly constrained so floods are observable)
net.addLink(s1, s3, bw=50)
net.addLink(s2, s3, bw=50)

info("*** Starting network\n")
net.build()
c0.start()
for s in (s1, s2, s3):
    s.start([c0])

info("*** Network ready. 6 hosts (10.0.0.1-6), 3 switches.\n")
info("*** Try: pingall then run traffic/ scripts from another xterm.\n")
CLI(net)
net.stop()

if __name__ == "__main__":
    setLogLevel("info")
    build()

```

## Appendix F: Traffic and Attack Tools

The port-scan attack simulator, which rapidly probes many ports on a target to imitate reconnaissance.

```
#!/usr/bin/env python3
"""
```

```
port_scan.py
=====
```

Simulates a TCP port scan - the "many tiny flows to many destinations" pattern the detector's PORT\_SCAN rule is designed to catch.

It rapidly attempts short-lived TCP connections to a wide range of ports on one or more targets. Each attempt sends only a handful of packets, so the resulting flows have very low packet counts spread across many distinct destination endpoints, which is exactly the port-scan signature.

Usage (inside a Mininet host, e.g. h1 python3 traffic/port\_scan.py 10.0.0.4):  
python3 port\_scan.py <target\_ip> [start\_port] [end\_port]

For a multi-host sweep, pass several targets:

```
python3 port_scan.py 10.0.0.2,10.0.0.4,10.0.0.5
"""
```

```
import socket
import sys
import time
```

```
def scan_target(ip, start, end):
    hit = 0
    for port in range(start, end + 1):
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(0.15)
        try:
            if s.connect_ex((ip, port)) == 0:
                hit += 1
        except Exception:
            pass
        finally:
            s.close()
    return hit
```

```
def main():
    if len(sys.argv) < 2:
        print("usage: port_scan.py <target_ip[,ip2,...]> [start] [end]")
        sys.exit(1)
    targets = sys.argv[1].split(",")
    start = int(sys.argv[2]) if len(sys.argv) > 2 else 1
    end = int(sys.argv[3]) if len(sys.argv) > 3 else 1024
    print(f"[scan] scanning {targets} ports {start}-{end}")
    t0 = time.time()
```

```
total_open = 0
for ip in targets:
    total_open += scan_target(ip, start, end)
print(f"[scan] done in {time.time()-t0:.1f}s, {total_open} open ports found")

if __name__ == "__main__":
    main()
```

## Appendix F (continued): Flood Simulator (flood.py)

The flood simulator, which sends traffic as fast as possible at a target to imitate a denial-of-service attack.

```
#!/usr/bin/env python3
"""
```

```
flood.py
```

```
=====
```

```
Simulates a packet-flood / DoS - the high-packet-rate pattern the detector's
FLOOD rule catches.
```

```
It blasts UDP datagrams at a target as fast as it can from a single source,
producing one (or a few) flows with a very high packets-per-second rate. This
is intentionally a controlled, self-contained flood between two emulated hosts;
it is only meant to be run inside the Mininet sandbox against your own target
host, never against real infrastructure.
```

```
Usage (inside a Mininet host, e.g. h1 python3 traffic/flood.py 10.0.0.4):
python3 flood.py <target_ip> [port] [duration_seconds]
```

```
If hping3 is installed you can get a heavier flood with:
```

```
hping3 --udp --flood -p 80 10.0.0.4
"""
```

```
import socket
import sys
import time
```

```
def main():
    if len(sys.argv) < 2:
        print("usage: flood.py <target_ip> [port] [duration]")
        sys.exit(1)
    target = sys.argv[1]
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 80
    duration = int(sys.argv[3]) if len(sys.argv) > 3 else 15

    payload = b"F" * 512
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    print(f"[flood] flooding {target}:{port} with UDP for {duration}s")
    end = time.time() + duration
    sent = 0
    try:
        while time.time() < end:
            try:
                s.sendto(payload, (target, port))
                sent += 1
            except Exception:
                pass
    except KeyboardInterrupt:
        pass
    rate = sent / duration if duration else 0
```

```
print(f"[flood] sent {sent} packets (~{rate:.0f} pkt/s)")

if __name__ == "__main__":
    main()
```

## Appendix G: Detection Engine Tests (test\_detection.py)

The automated unit tests that validate the detection engine's behaviour on synthetic traffic, covering each attack type, the EMA smoothing, the sustained-window hysteresis, and the important normal-traffic case.

```
"""
test_detection.py
Unit tests for the anomaly-detection engine (controller/detection.py).

These run WITHOUT Ryu/Mininet - they exercise the detection logic directly by
feeding it synthetic FlowRecords, so they can be run on any machine with:

    python3 -m pytest tests/test_detection.py -v
    or simply
    python3 tests/test_detection.py
"""
import os
import sys
import time

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "controller"))

from detection import (AnomalyDetector, DetectionConfig, FlowRecord) # noqa: E402

def _flow(src, dst, dport, pkts=1, byts=64, prate=0.0, brate=0.0,
          sport=40000, proto="TCP", dur=0.5):
    return FlowRecord(src_ip=src, dst_ip=dst, src_port=sport, dst_port=dport,
                      protocol=proto, packet_count=pkts, byte_count=byts,
                      packet_rate=prate, byte_rate=brate, duration_sec=dur)

def test_port_scan_detected():
    cfg = DetectionConfig(scan_distinct_targets=15, scan_max_pkts_per_flow=3)
    det = AnomalyDetector(cfg)
    # one source hitting 20 distinct ports on one host, 1 packet each
    flows = [_flow("10.0.0.1", "10.0.0.2", 1000 + i, pkts=1) for i in range(20)]
    alerts = det.process(flows)
    cats = [a.category for a in alerts]
    assert "PORT_SCAN" in cats, f"expected PORT_SCAN, got {cats}"
    scan = next(a for a in alerts if a.category == "PORT_SCAN")
    assert scan.evidence["distinct_endpoints"] >= 15
    print("PASS test_port_scan_detected:", scan.description)

def test_normal_traffic_no_scan():
    cfg = DetectionConfig(scan_distinct_targets=15)
    det = AnomalyDetector(cfg)
    # a normal host talking to 3 services with substantial packet counts
    flows = [
        _flow("10.0.0.1", "10.0.0.2", 80, pkts=500, byts=600000),
        _flow("10.0.0.1", "10.0.0.3", 443, pkts=800, byts=900000),
```

```

        _flow("10.0.0.1", "10.0.0.4", 22, pkts=120, byts=40000),
    ]
    alerts = det.process(flows)
    assert not any(a.category == "PORT_SCAN" for a in alerts), \
        "normal traffic wrongly flagged as scan"
    print("PASS test_normal_traffic_no_scan")

def test_flood_detected_per_flow():
    # with the default 3-window hysteresis, a sustained high-rate flow fires
    cfg = DetectionConfig(flood_pkt_rate=500) # sustained windows = 3 default
    det = AnomalyDetector(cfg)
    flows = [_flow("10.0.0.9", "10.0.0.2", 80, pkts=90000,
                   prate=9000.0, brate=5_000_000.0)]
    alerts = []
    for _ in range(cfg.flood_sustained_windows): # sustained attack
        alerts = det.process(flows)
    assert any(a.category == "FLOOD" for a in alerts), "flood not detected"
    flood = next(a for a in alerts if a.category == "FLOOD")
    assert flood.severity == "HIGH"
    assert flood.evidence["sustained_windows"] >= cfg.flood_sustained_windows
    print("PASS test_flood_detected_per_flow:", flood.description)

def test_flood_detected_aggregate():
    cfg = DetectionConfig(flood_pkt_rate=5000,
                          flood_aggregate_pkt_rate=1000)
    det = AnomalyDetector(cfg)
    # 10 modest flows that individually stay under the per-flow bar but whose
    # smoothed sum crosses the aggregate bar
    flows = [_flow("10.0.0.9", f"10.0.0.{i}", 80, prate=150.0)
              for i in range(2, 12)]
    alerts = []
    for _ in range(cfg.flood_sustained_windows):
        alerts = det.process(flows)
    flood = [a for a in alerts if a.category == "FLOOD"]
    assert flood and flood[0].evidence["trigger"] == "aggregate"
    print("PASS test_flood_detected_aggregate:", flood[0].description)

def test_flood_hysteresis_ignores_single_spike():
    # a single brief cycle just over the per-flow threshold must NOT raise a
    # flood, because the sustained-window hysteresis is not satisfied and the
    # smoothed rate falls straight back below threshold
    cfg = DetectionConfig(flood_pkt_rate=500, flood_aggregate_pkt_rate=1000,
                          flood_sustained_windows=3)
    det = AnomalyDetector(cfg)
    normal = [_flow("10.0.0.5", "10.0.0.2", 80, prate=50.0)]
    spike = [_flow("10.0.0.5", "10.0.0.2", 80, prate=600.0)]
    alerts = []
    alerts += det.process(normal)
    alerts += det.process(spike) # single transient over-threshold cycle
    alerts += det.process(normal)
    alerts += det.process(normal)

```

```

assert not any(a.category == "FLOOD" for a in alerts), \
    "a single transient spike should not trigger a flood alert"
print("PASS test_flood_hysteresis_ignores_single_spike")

def test_flood_ema_smooths_toward_rate():
    # the smoothed EMA should climb toward the true rate over successive polls
    cfg = DetectionConfig(flood_pkt_rate=500, ema_alpha=0.4,
                          flood_sustained_windows=3)
    det = AnomalyDetector(cfg)
    flow = [_flow("10.0.0.7", "10.0.0.2", 80, prate=1000.0)]
    for _ in range(cfg.flood_sustained_windows):
        det.process(flow)
    ema = det.source_metrics()["10.0.0.7"]["ema_pkt_rate"]
    assert 500.0 < ema <= 1000.0, f"EMA did not converge sensibly: {ema}"
    print(f"PASS test_flood_ema_smooths_toward_rate: EMA={ema:.0f} pkt/s")

def test_volume_anomaly_after_baseline():
    cfg = DetectionConfig(baseline_min_samples=5, volume_multiplier=8,
                          volume_abs_floor_bps=1_000_000,
                          alert_cooldown_seconds=0)
    det = AnomalyDetector(cfg)
    # establish a calm baseline of ~0.2 MB/s over several cycles
    for _ in range(6):
        det.process([_flow("10.0.0.1", "10.0.0.2", 80, brate=200_000.0)])
    # now a spike to 5 MB/s (25x baseline)
    alerts = det.process([_flow("10.0.0.1", "10.0.0.2", 80,
                               brate=5_000_000.0)])
    assert any(a.category == "VOLUME" for a in alerts), "volume spike missed"
    vol = next(a for a in alerts if a.category == "VOLUME")
    assert vol.evidence["ratio"] >= 8
    print("PASS test_volume_anomaly_after_baseline:", vol.description)

def test_cooldown_suppresses_duplicates():
    cfg = DetectionConfig(flood_pkt_rate=500, alert_cooldown_seconds=60,
                          flood_sustained_windows=1)
    det = AnomalyDetector(cfg)
    f = [_flow("10.0.0.9", "10.0.0.2", 80, prate=9000.0)]
    first = det.process(f)
    second = det.process(f) # immediately again -> should be suppressed
    assert any(a.category == "FLOOD" for a in first)
    assert not any(a.category == "FLOOD" for a in second), \
        "cooldown failed to suppress duplicate flood alert"
    print("PASS test_cooldown_suppresses_duplicates")

def _run_all():
    tests = [v for k, v in sorted(globals().items()) if k.startswith("test_")]
    passed = 0
    for t in tests:
        t()
        passed += 1

```



```
print(f"\n{passed}/{len(tests)} detection tests passed.")

if __name__ == "__main__":
    _run_all()
```

## Appendix H: Evaluation Metrics Script (evaluate.py)

The offline evaluation tool that reads the controller's CSV logs and computes precision, recall, F1, accuracy, detection latency, mitigation latency, and recovery time for a controlled attack.

```
#!/usr/bin/env python3
"""
evaluate.py
=====
Compute quantitative detection metrics from the CSV logs the controller writes
(logs/events.csv and logs/rates.csv), given the attacker's IP and the attack
window.

It reports, in the style of a security-evaluation table:

* Precision, Recall, F1 and Accuracy    (per polling interval, for the attacker)
* Detection latency                     (first FLOOD alert - attack start)
* Mitigation latency                    (first mitigation - attack start)
* Recovery time                         (mitigation cleared - attack end)
* Peak attack packet rate
```

Ground truth is defined per polling interval for the attacker source: an interval is labelled "attack" if its timestamp falls inside the attack window. A "detection" for that interval means a FLOOD alert for the attacker was raised at or before that interval while the attack was ongoing, or a mitigation was active during it. Comparing the two over every interval yields the confusion matrix the precision/recall figures are built from.

### Usage

```
-----
# attack window given as clock times (same day as the logs):
python3 analysis/evaluate.py --attacker 10.0.0.1 \
    --attack-start "14:03:10" --attack-end "14:04:05"

# or as epoch seconds:
python3 analysis/evaluate.py --attacker 10.0.0.1 \
    --attack-start 1737465790 --attack-end 1737465845

# or let it infer the window from when the attacker's raw rate was high:
python3 analysis/evaluate.py --attacker 10.0.0.1 --auto
"""

import argparse
import csv
import os
import sys
import time

def _parse_when(value, day_ref):
    """Accept an epoch value or a HH:MM:SS clock time on the log's date."""
    if value is None:
        return None
```

```

try:
    return float(value)                    # already epoch seconds
except ValueError:
    pass
# treat as HH:MM:SS on the same local day as the first log timestamp
lt = time.localtime(day_ref)
for fmt in ("%H:%M:%S", "%H:%M"):
    try:
        t = time.strptime(value, fmt)
        return time.mktime((lt.tm_year, lt.tm_mon, lt.tm_mday,
                               t.tm_hour, t.tm_min, t.tm_sec,
                               0, 0, -1))
    except ValueError:
        continue
raise SystemExit(f"could not parse time: {value!r}")

def load_rates(path, attacker):
    rows = []
    with open(path, newline="") as f:
        for r in csv.DictReader(f):
            if r["src_ip"] != attacker:
                continue
            rows.append({
                "ts": float(r["timestamp"]),
                "pkt_rate": float(r["agg_pkt_rate"]),
                "ema": float(r.get("ema_pkt_rate", 0) or 0),
                "streak": int(float(r.get("flood_streak", 0) or 0)),
            })
    rows.sort(key=lambda x: x["ts"])
    return rows

def load_events(path, attacker):
    alerts, mitigations, recoveries = [], [], []
    with open(path, newline="") as f:
        for r in csv.DictReader(f):
            if r["src_ip"] != attacker:
                continue
            ts = float(r["timestamp"])
            ev = r["event"]
            if ev == "ALERT" and r["category"] == "FLOOD":
                alerts.append(ts)
            elif ev == "MITIGATION":
                mitigations.append(ts)
            elif ev == "RECOVERY":
                recoveries.append(ts)
    return sorted(alerts), sorted(mitigations), sorted(recoveries)

def main():
    ap = argparse.ArgumentParser(description="SDN detection metrics")
    ap.add_argument("--attacker", required=True, help="attacker source IP")
    ap.add_argument("--logs", default="logs", help="log directory")

```

```

ap.add_argument("--attack-start", help="epoch or HH:MM:SS")
ap.add_argument("--attack-end", help="epoch or HH:MM:SS")
ap.add_argument("--auto", action="store_true",
                help="infer the attack window from the attacker's raw rate")
ap.add_argument("--auto-threshold", type=float, default=200.0,
                help="raw pkt/s that marks 'attacking' in --auto mode")
ap.add_argument("--report", default=None,
                help="also write the report to this file")
args = ap.parse_args()

rates_path = os.path.join(args.logs, "rates.csv")
events_path = os.path.join(args.logs, "events.csv")
for p in (rates_path, events_path):
    if not os.path.exists(p):
        raise SystemExit(f"missing log file: {p} (run the controller first)")

rates = load_rates(rates_path, args.attacker)
if not rates:
    raise SystemExit(f"no rate samples for {args.attacker} in {rates_path}")
alerts, mitigations, recoveries = load_events(events_path, args.attacker)

day_ref = rates[0]["ts"]
if args.auto:
    hot = [r["ts"] for r in rates if r["pkt_rate"] >= args.auto_threshold]
    if not hot:
        raise SystemExit("no intervals exceeded --auto-threshold; give "
                        "--attack-start/--attack-end explicitly")
    start, end = min(hot), max(hot)
else:
    if not (args.attack_start and args.attack_end):
        raise SystemExit("give --attack-start and --attack-end, or --auto")
    start = _parse_when(args.attack_start, day_ref)
    end = _parse_when(args.attack_end, day_ref)

# per-interval confusion matrix for the attacker
first_alert = alerts[0] if alerts else None

# when did blocking finally clear? (first recovery after the attack, or an
# idle-timeout estimate). Benign intervals inside (end, recovery_end] are a
# transitional recovery period and are scored separately, not as FPs.
after_rec = [rt for rt in recoveries if rt >= end]
recovery_end = after_rec[0] if after_rec else (end + 30.0)

def mitigation_active(ts):
    return any(m <= ts <= m + _mitigation_span(recoveries, m)
              for m in mitigations)

tp = fp = fn = tn = skipped = 0
for r in rates:
    ts = r["ts"]
    is_attack = start <= ts <= end
    detected = ((first_alert is not None and ts >= first_alert
                and is_attack) or mitigation_active(ts))
    if is_attack:

```

```

        if detected:
            tp += 1
        else:
            fn += 1
    else:
        # benign interval
        if end < ts <= recovery_end:
            skipped += 1          # transitional recovery, not scored
            continue
        if detected:
            fp += 1
        else:
            tn += 1

precision = tp / (tp + fp) if (tp + fp) else 0.0
recall = tp / (tp + fn) if (tp + fn) else 0.0
f1 = (2 * precision * recall / (precision + recall)
      if (precision + recall) else 0.0)
accuracy = (tp + tn) / (tp + tn + fp + fn) if rates else 0.0

detection_latency = (first_alert - start) if first_alert else None
mitigation_latency = (mitigations[0] - start) if mitigations else None
recovery_time = None
if recoveries:
    after = [rt for rt in recoveries if rt >= end]
    recovery_time = (after[0] - end) if after else None
peak_rate = max((r["pkt_rate"] for r in rates), default=0.0)

lines = []
add = lines.append
add("=" * 58)
add(" SDN DETECTION EVALUATION")
add("=" * 58)
add(f" Attacker source          : {args.attacker}")
add(f" Attack window              : "
    f"{time.strftime('%H:%M:%S', time.localtime(start))} - "
    f"{time.strftime('%H:%M:%S', time.localtime(end))} "
    f"({end - start:.1f}s)")
add(f" Polling intervals scored : {len(rates)}")
if skipped:
    add(f" (recovery-window intervals excluded from scoring: {skipped})")
add("-" * 58)
add(f" Confusion matrix   TP={tp}  FP={fp}  FN={fn}  TN={tn}")
add(f" Precision           : {precision * 100:5.1f}%")
add(f" Recall (detection rate) : {recall * 100:5.1f}%")
add(f" F1 score            : {f1 * 100:5.1f}%")
add(f" Accuracy            : {accuracy * 100:5.1f}%")
add("-" * 58)
add(f" Detection latency      : "
    + (f"{detection_latency:5.1f}s" if detection_latency is not None
      else " n/a (no alert)"))
add(f" Mitigation latency     : "
    + (f"{mitigation_latency:5.1f}s" if mitigation_latency is not None
      else " n/a (no mitigation)"))

```

```

add(f" Recovery time          : "
    + (f"{recovery_time:5.1f}s" if recovery_time is not None
        else " n/a (still blocked / not logged)"))
add(f" Peak attack rate      : {peak_rate:7.0f} pkt/s")
add("=" * 58)
report = "\n".join(lines)
print(report)

if args.report:
    with open(args.report, "w") as f:
        f.write(report + "\n")
    print(f"\n(report written to {args.report})")

def _mitigation_span(recoveries, m):
    """How long a mitigation installed at time m stayed active (until the next
    recovery event at or after m); default to 30s if none was logged."""
    later = [rt for rt in recoveries if rt >= m]
    return (later[0] - m) if later else 30.0

if __name__ == "__main__":
    main()

```